

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

**«Белорусский государственный университет информатики и
радиоэлектроники»**

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №1

по дисциплине

**«Современные технологии программирования масштабируемых
приложений»**

**«Микросервис ввода и модификации данных о клиентах банка. Модуль
«Клиенты»»**

Выполнил:

Лебедев Артем Владимирович
магистрант группы 556301

Преподаватель:

Хмелёва Ангелина Владимировна
кандидат технических наук
доцент кафедры ПОИТ

Минск, 2026 г.

СОДЕРЖАНИЕ

1	Цель работы	1
2	Задание	2
3	Проектирование	3
3.1	Архитектура	4
3.2	Схема данных	4
3.3	REST-интерфейс	6
4	Реализация	6
4.1	Структура проекта	7
4.2	Web-клиент	7
4.3	Двойная проверка маски и обязательности	7
4.4	Уникальность паспорта и идентификационного номера	10
4.5	Локализация	10
5	Тестирование	11
5.1	Уровни тестов	12
5.2	Программа защиты в виде автоматических тестов	12
5.3	Результаты прогона	14
6	Результаты работы	14
	Заключение	21
	Список использованных источников	22
	Приложение А Исходный код микросервиса «Клиенты»	23

1 ЦЕЛЬ РАБОТЫ

Разработать учебную подсистему ввода и модификации данных о клиентах условного банка: спроектировать схему данных со справочниками, реализовать микросервис с REST-интерфейсом на Java и Spring, web-клиент с формами «Список клиентов» и добавления, редактирования и удаления клиента, обеспечить проверку вводимых значений дважды — в браузере и в сервисе — и покрыть разработанное ПО модульными, интеграционными и функциональными тестами. Дополнительно к заданию интерфейс и сообщения сервиса сделаны на трёх языках: русском, английском и белорусском.

2 ЗАДАНИЕ

Работа выполнена по варианту № 4. Номер варианта определяет состав полей клиента: вариант чётный, поэтому в форму входит поле «Пол» и не входят «Место работы» и «Должность»; $4 \bmod 3 = 1$, поэтому из трёх полей «Город прописки», «Адрес прописки» и «Военнообязанный» реализуется первое. Итоговый состав полей приведён в таблице 2.1.

Таблица 2.1 – Поля клиента для варианта № 4

Поле	Тип	Обяз.	Контроль значения
Фамилия, имя, отчество	текстовый	да	только буквы
Дата рождения	дата	да	существующая дата, не позднее сегодня
Пол	radio	да	мужской или женский
Серия паспорта	текстовый	да	две латинские буквы
№ паспорта	текстовый с маской	да	7 цифр, уникален вместе с серией
Кем выдан	текстовый	да	не пусто
Дата выдачи	дата	да	не раньше даты рождения
Идентификационный номер	текстовый с маской	да	3140301A001PB5, уникален
Место рождения	текстовый	да	не пусто
Город проживания	список	да	значение справочника
Адрес проживания	текстовый	да	не пусто
Телефон домашний	текстовый с маской	нет	293-88-44
Телефон мобильный	текстовый с маской	нет	+375 (29) 314-15-92
E-mail	текстовый	нет	формат адреса
Город прописки	список	да	значение справочника
Семейное положение	список	да	значение справочника
Гражданство	список	да	значение справочника
Инвалидность	список	да	значение справочника
Пенсионер	checkbox	да	да или нет
Ежемесячный доход	денежный	нет	неотрицательная сумма

Требования задания:

1. разработать схему данных со справочниками для элементов типа «Список» и реализовать её в СУБД, работающей в памяти; база заполняется автоматически при старте;
2. создать микросервис для REST-подключения к базе данных;
3. разработать web-клиент с формой «Список клиентов», отсортированной по фамилиям, и формой добавления, удаления и редактирования клиентов;
4. проверять значения каждого поля дважды — на клиенте и в REST;
5. добавить в базу не менее пяти клиентов, включая себя;
6. протестировать ПО на WebDriver с базой H2 в памяти по пунктам программы защиты.

3 ПРОЕКТИРОВАНИЕ

3.1 Архитектура

Модуль реализован как самостоятельный микросервис: у сервиса `client-service` своя база данных, собственный REST-интерфейс и собственный web-клиент, который раздаётся тем же процессом как статический ресурс. Остальные модули банка (депозиты, кредиты) получают сведения о клиентах только через REST, поэтому сервис можно развернуть, остановить и масштабировать независимо от них. Условный банк в системе называется BankEt. Структура сервиса показана на рисунке 3.1.

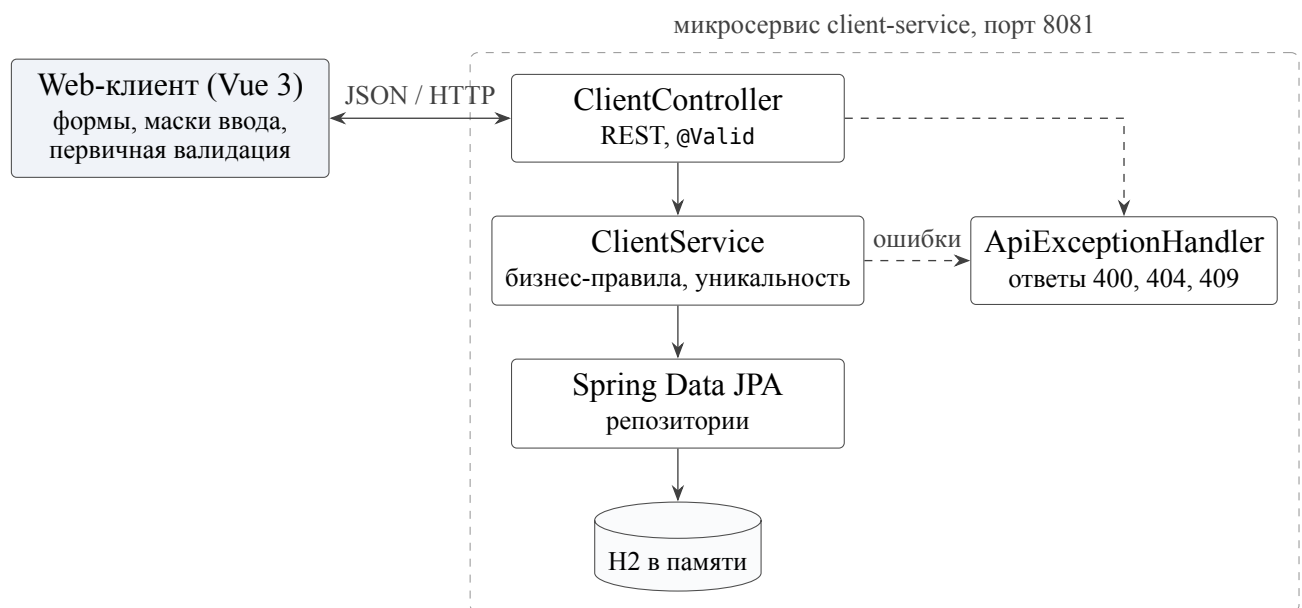


Рисунок 3.1 – Структура микросервиса «Клиенты»

Использованные технологии: Java 17, Spring Boot 3.5 (Spring MVC, Spring Data JPA, Bean Validation), H2 в режиме in-memory, Vue 3 на стороне браузера, JUnit 5, Mockito, MockMvc и Selenium WebDriver в тестах, Maven в качестве системы сборки. Библиотека Vue подключается из webjar и раздаётся самим сервисом, поэтому для запуска не нужны ни Node.js, ни доступ в интернет.

3.2 Схема данных

Каждое поле типа «Список» вынесено в отдельный справочник: `city` (используется дважды — для города проживания и города прописки), `marital_status`, `citizenship` и `disability`. Таблица `client` ссылается на справочники внешними ключами (рисунок 3.2). Пол хранится символом M/F с ограничением `check`, поскольку набор значений фиксирован и справочник для него не нужен. Наименование в каждом справочнике хранится на трёх языках: `name` — русское, `name_en` и `name_be` — английское и белорусское.

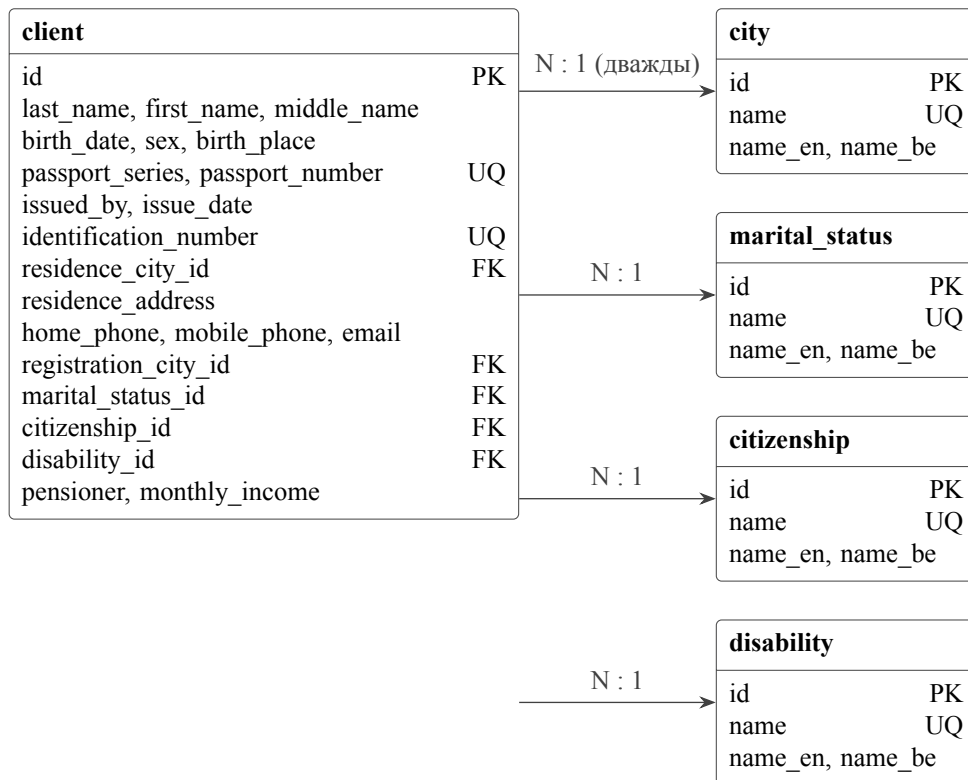


Рисунок 3.2 – Схема данных модуля «Клиенты»

Схема создаётся сценарием `schema.sql`, справочники и шесть тестовых клиентов (включая автора работы) загружаются сценарием `data.sql` при каждом старте приложения. Hibernate работает в режиме `ddl-auto=validate`: он не создаёт таблицы сам, а сверяет сущности с явно описанной схемой и не даст сервису стартовать при расхождении. Ограничения уникальности вынесены в саму базу (листинг 3.1) — это последний рубеж защиты от дублей, который срабатывает даже в обход сервиса.

Листинг 3.1 – Фрагмент `schema.sql`: ссылки на справочники и уникальность

```

create table client (
    id                bigint generated by default as identity primary key,
    last_name         varchar(60) not null,
    ...
    sex              varchar(1) not null check (sex in ('M', 'F')),
    residence_city_id bigint not null references city,
    registration_city_id bigint not null references city,
    marital_status_id bigint not null references marital_status,
    citizenship_id    bigint not null references citizenship,
    disability_id     bigint not null references disability,
    pensioner         boolean not null,
    monthly_income    decimal(14, 2),
    constraint uq_client_passport unique (passport_series, passport_number),
    constraint uq_client_identification unique (identification_number)
);

```

3.3 REST-интерфейс

Сервис предоставляет шесть операций (таблица 3.1). Данные передаются в JSON, даты — текстом в формате ДД.ММ.ГГГГ: это позволяет серверу самому решать, является ли присланная строка датой, и возвращать ошибку конкретного поля, а не общий сбой разбора запроса. Любая ошибка возвращается в едином виде — общее сообщение и словарь «поле — сообщение», по которому web-клиент подсвечивает поля формы. Язык сообщений и наименований справочников определяется заголовком запроса `Accept-Language`.

Таблица 3.1 – REST-интерфейс микросервиса «Клиенты»

Метод	Адрес	Назначение и коды ответа
GET	/api/clients	список клиентов, отсортированный по фамилии; 200
GET	/api/clients/{id}	данные клиента; 200, 404
POST	/api/clients	добавление; 201, 400 — ошибка валидации, 409 — дубль
PUT	/api/clients/{id}	изменение; 200, 400, 404, 409
DELETE	/api/clients/{id}	удаление; 204, 404
GET	/api/dictionaries	содержимое четырёх справочников для списков формы

4 РЕАЛИЗАЦИЯ

4.1 Структура проекта

Сервис является модулем многомодульного Maven-проекта банка. Код разложен по слоям (листинг 4.1); общие для всех web-клиентов стили, маски ввода, выбор языка и обёртка над fetch вынесены в модуль `ui-kit`, а серверная часть локализации — в пакет `il8n` общей библиотеки `bank-common`.

Листинг 4.1 – Структура модуля `client-service`

```
src/main/java/by/bsuir/bank/client/
  domain/      Client, справочники City, MaritalStatus, Citizenship, Disability
  repository/  репозитории Spring Data JPA
  dto/         ClientRequest (форма с правилами валидации), ClientResponse, ApiError
  validation/  Masks – маски полей, Dates и @DateText – строгая проверка дат
  service/     ClientService – уникальность, справочники, сохранение
  web/         ClientController, DictionaryController, ApiExceptionHandler
src/main/resources/
  schema.sql   схема данных          data.sql   справочники и тестовые клиенты
  messages.properties, messages_en.properties, messages_be.properties – тексты сервиса
  static/      index.html, app.js – web-клиент на Vue 3, il8n.js – его словарь
src/test/java/  модульные (*Test), интеграционные и функциональные (*IT) тесты
```

4.2 Web-клиент

Web-клиент — одностраничное приложение на Vue 3 с двумя формами. Форма «Список клиентов» запрашивает `GET /api/clients` и показывает таблицу; сортировку по фамилии выполняет сервис, чтобы порядок был одинаковым для любого потребителя REST. Форма клиента используется и для добавления, и для редактирования; удаление доступно из списка и из формы.

Форма построена по описанию полей: для каждого поля в одном месте заданы тип, обязательность, маска ввода и функция проверки, а разметка и валидация генерируются по этому описанию; подписи берутся из словаря по имени поля. Добавление поля сводится к одной строке, и расхождение между тем, что нарисовано, и тем, что проверяется, становится невозможным. Маска (`#` — цифра, `A` — латинская буква) применяется при каждом нажатии клавиши: недопустимые символы отбрасываются, разделители расставляются автоматически, поэтому в поле даты физически нельзя набрать текст.

4.3 Двойная проверка маски и обязательности

Проверка значений выполняется дважды. Первичная проверка в браузере нужна для удобства: пользователь сразу видит сообщения у полей и не ждёт ответа сервера. Но она не защищает базу — запрос можно отправить в обход формы, поэтому сервис повторяет все проверки, и именно он является источником истины.

Первичная проверка на клиенте показана в листинге 4.2. Полу жирным выделен фрагмент, который для каждого поля проверяет обязательность (признак `required`) и соответствие маске (функция `check` с регулярным выражением).

Листинг 4.2 – Первичная проверка в web-клиенте (`static/app.js`)

```
const SECTIONS = [
  ...
  { title: 'passport', fields: [
    { name: 'passportSeries', required: true, mask: 'AA', placeholder: 'MP',
      check: v => /^[A-Z]{2}$/.test(v) || t('check.passportSeries') },
    { name: 'passportNumber', required: true, mask: '#####', placeholder: '1234567',
      check: v => /^[d{7}$/.test(v) || t('check.passportNumber') },
    { name: 'identificationNumber', required: true, mask: '#####A###AA#', placeholder: '
3140301A001PB5',
      check: v => /^[d{7}[A-Z]\d{3}[A-Z]{2}\d$/.test(v) || t('check.identificationNumber') },
    ...
  ]
};

/* Проверка формы перед отправкой: возвращает { имяПоля: сообщение }. Пустой объект – ошибок
нет. */
function validate(form) {
  const errors = {};
  for (const field of FIELDS) {
    const raw = form[field.name];
    const value = typeof raw === 'string' ? raw.trim() : raw;
    const empty = value === null || value === undefined || value === '';
    if (empty) {
      if (field.required) errors[field.name] = t('common.required');
      continue;
    }
    const result = field.check ? field.check(value, form) : true;
    if (result !== true) errors[field.name] = result;
  }
  return errors;
}
```

Проверка в сервисе (листинг 4.3) описана декларативно аннотациями Bean Validation на записи `ClientRequest`; контроллер помечает параметр аннотацией `@Valid`, и до бизнес-логики запрос с ошибками не доходит. Полу жирным выделены ограничения обязательности (`@NotBlank`, `@NotNull`) и маски (`@Pattern`, `@DateText`). Регулярные выражения масок собраны в классе `Masks` и по содержанию совпадают с выражениями web-клиента. Сообщения заданы ключами в фигурных скобках: текст подставляется из словаря на языке запроса.

Листинг 4.3 – Серверная проверка: запись `ClientRequest` и маски `Masks`

```
public record ClientRequest(

    @NotBlank(message = REQUIRED) @Size(max = 60, message = TOO_LONG)
    @Pattern(regexp = Masks.NAME, message = LETTERS_ONLY)
    String lastName,
    ...
    @NotBlank(message = REQUIRED) @DateText
    String birthDate,
```

```

        @NotBlank(message = REQUIRED)
        @Pattern(regexp = Masks.PASSPORT_SERIES, message = "{validation.passportSeries}")
        String passportSeries,

        @NotBlank(message = REQUIRED)
        @Pattern(regexp = Masks.PASSPORT_NUMBER, message = "{validation.passportNumber}")
        String passportNumber,

        ...

        @NotBlank(message = REQUIRED)
        @Pattern(regexp = Masks.IDENTIFICATION_NUMBER, message = "{validation.
identificationNumber}")
        String identificationNumber,

        ...

        @Pattern(regexp = Masks.MOBILE_PHONE, message = "{validation.mobilePhone}")
        String mobilePhone,

        @NotNull(message = REQUIRED)
        Long residenceCityId,

        ...
    ) { ... }

    public final class Masks {
        public static final String NAME = "[A-Яа-яЁёА-Зз-з]+(?:[-' ][A-Яа-яЁёА-Зз-з]+)*$";
        public static final String PASSPORT_SERIES = "[A-Z]{2}$";
        public static final String PASSPORT_NUMBER = "\\d{7}$";
        public static final String IDENTIFICATION_NUMBER = "\\d{7}[A-Z]\\d{3}[A-Z]{2}\\d$";
        public static final String HOME_PHONE = "^$|^\\d{3}-\\d{2}-\\d{2}$";
        public static final String MOBILE_PHONE = "^$|^\\+375 \\((25|29|33|44)\\) \\d{3}-\\d{2}-\\d{2}$";
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public ClientResponse create(@Valid @RequestBody ClientRequest request) {
        return service.create(request);
    }
}

```

Три детали серверной проверки заслуживают отдельного пояснения.

Пробел вместо имени. Компактный конструктор записи отбрасывает пробелы по краям всех строк ещё до проверок, поэтому строка из одного пробела превращается в пустую и отклоняется как незаполненное обязательное поле.

Несуществующая дата. Стандартный разбор даты в Java по умолчанию работает в режиме SMART и молча превращает 31 февраля в 28-е. Для строгой проверки формат создаётся с `ResolverStyle.STRICT` (листинг 4.4): и 31.02.2015, и 29.02.2015 отвергаются, а 29.02.2016 принимается. Собственное ограничение `@DateText` дополнительно требует, чтобы дата лежала между 01.01.1900 и сегодняшним днём.

Листинг 4.4 – Строгий разбор даты (validation/Dates.java)

```

public static final DateTimeFormatter FORMAT =
    DateTimeFormatter.ofPattern("dd.MM.uuuu").withResolverStyle(ResolverStyle.STRICT);

public static Optional<LocalDate> parse(String text) {

```

```

if (text == null) {
    return Optional.empty();
}
try {
    return Optional.of(LocalDate.parse(text.trim(), FORMAT));
} catch (DateTimeParseException e) {
    return Optional.empty();
}
}

```

Необязательные поля. Для телефонов и e-mail маска допускает пустую строку (^\$|...), а сервис сохраняет пустое значение как NULL: запись клиента только с обязательными полями выполняется без ошибок.

4.4 Уникальность паспорта и идентификационного номера

Маска не может обнаружить повторный ввод того же клиента — это правило проверяется в сервисе обращением к базе (листинг 4.5). При редактировании собственная запись клиента из сравнения исключается, иначе его нельзя было бы сохранить без смены паспорта. Нарушение возвращается с кодом 409 и сообщениями у полей паспорта и идентификационного номера. Если два одинаковых запроса придут одновременно и оба пройдут проверку, второй будет остановлен ограничением уникальности в базе; обработчик ошибок превращает `DataIntegrityViolationException` в тот же ответ 409.

Листинг 4.5 – Проверка уникальности (service/ClientService.java)

```

private void checkUnique(ClientRequest request, Long selfId) {
    Map<String, String> errors = new LinkedHashMap<>();
    clients.findByPassportSeriesAndPassportNumber(request.passportSeries(), request.
        passportNumber())
        .filter(other -> !other.getId().equals(selfId))
        .ifPresent(other -> {
            errors.put("passportSeries", Messages.get("client.duplicatePassport"));
            errors.put("passportNumber", Messages.get("client.duplicatePassport"));
        });
    clients.findByIdentificationNumber(request.identificationNumber())
        .filter(other -> !other.getId().equals(selfId))
        .ifPresent(other -> errors.put("identificationNumber", Messages.get("client.
            duplicateIdentification")));
    if (!errors.isEmpty()) {
        throw new FieldErrorException(HttpStatus.CONFLICT, Messages.get("client.duplicate"),
            errors);
    }
}

```

4.5 Локализация

Язык выбирается переключателем в шапке любого web-клиента. Выбор хранится в cookie `bank_lang`: cookie не зависит от порта, поэтому язык един для всех web-клиентов системы. Перевод выполняется в трёх местах (таблица 4.1).

Таблица 4.1 – Где переводятся тексты

Что	Где хранится перевод	Как выбирается язык
Подписи, кнопки, первичная проверка	словарь <code>il18n.js</code> web-клиента	функция <code>Bank.t</code> по cookie
Сообщения сервиса	<code>messages*.properties</code>	заголовок <code>Accept-Language</code>
Значения справочников	столбцы <code>name</code> , <code>name_en</code> , <code>name_be</code>	заголовок <code>Accept-Language</code>

Web-клиент при каждом REST-вызове передаёт выбранный язык в заголовке `Accept-Language` (листинг 4.6). В сервисе язык запроса определяет стандартный `AcceptHeaderLocaleResolver` с тремя поддерживаемыми языками и русским по умолчанию, а тексты выдаёт `MessageSource` из файлов `messages.properties` (русский), `messages_en.properties` и `messages_be.properties`. Тот же источник использует Bean Validation, поэтому ключ `{validation.required}` в аннотации превращается в «Обязательное поле», «Required field» или «Абавязковае поле».

Листинг 4.6 – Выбор текста и передача языка сервису (`ui-kit/bank.js`)

```
lang: (document.cookie.match(/(?:^|; )bank_lang=(ru|en|be)/) || []).length || 'ru',

t(key, ...args) {
  const text = Bank.messages[Bank.lang][key] ?? Bank.messages.ru[key] ?? key;
  return text.replace(/\{(\d+)\}/g, (match, index) => args[index] ?? '');
},

async api(method, url, body) {
  const options = { method, headers: { 'Accept': 'application/json', 'Accept-Language': Bank.lang } };
  ...
}
```

Значения справочников — это данные, а не тексты программы, поэтому их переводы хранятся в базе рядом с русским наименованием. Сущность справочника отдаёт наименование на языке запроса (листинг 4.7), и в ответе REST всегда одно поле `name`: потребителю не нужно знать о существовании переводов. Данные самого клиента — ФИО, адрес — не переводятся.

Листинг 4.7 – Наименование справочника на языке запроса

```
// DictionaryItem – общий предок справочников
public String getName() {
  return Messages.pick(name, nameEn, nameBe);
}

// Messages – тексты на языке пользователя
public static String pick(String ru, String en, String be) {
  String language = locale().getLanguage();
  String value = "en".equals(language) ? en : "be".equals(language) ? be : ru;
  return value == null || value.isBlank() ? ru : value;
}
```

5 ТЕСТИРОВАНИЕ

5.1 Уровни тестов

Тесты образуют пирамиду из трёх уровней (таблица 5.1). Модульные тесты запускаются на фазе `test` и не поднимают Spring; интеграционные и функциональные (имена классов оканчиваются на `IT`) выполняются плагином Failsafe на фазе `verify`. Все уровни используют базу H2 в памяти; у каждого класса тестов своя база, поэтому тесты не влияют друг на друга.

Таблица 5.1 – Тесты микросервиса «Клиенты»

Класс тестов	Тестов	Что проверяется
<i>Модульные — без Spring и базы данных</i>		
DateTextValidatorTest	16	существующие и несуществующие даты
ClientRequestValidationTest	49	маски и обязательность каждого поля
ClientServiceTest	8	уникальность и справочники на заглушках Mockito
<i>Интеграционные — сервис целиком и H2</i>		
ClientApiIT	39	REST через MockMvc, серверная валидация, состояние таблицы
ClientLanguageIT	9	сообщения и справочники на трёх языках
<i>Функциональные — браузер через WebDriver</i>		
ClientUiIT	37	сценарии программы защиты и переключение языка в web-клиенте
Всего	158	

5.2 Программа защиты в виде автоматических тестов

Функциональные тесты повторяют пункты, по которым проводится защита работы (таблица 5.2). Под данными «Иванов Иван Иванович» в задании понимаются данные студента; в тестах пунктов 2 и 3 новый клиент вводится с паспортом и идентификационным номером клиента Лебедевич из начального заполнения базы. Каждый тест после действий в браузере выполняет SQL-запрос к таблице `client` — проверку «прямо в базе».

Таблица 5.2 – Соответствие пунктов защиты и тестов

Пункт	Сценарий и ожидаемый результат	Тест ClientUiIT
1	те же данные ещё раз — ошибка, в базе один клиент	sameClientTwice
2	другой клиент с тем же паспортом — ошибка	anotherClientWithSamePassport
3	другой клиент с тем же идентификационным номером — ошибка	...SameIdentificationNumber
4	фамилия «1234» — ошибка, фамилии 1234 в базе нет	digitsInsteadOfLastName
5	пробел вместо имени — ошибка, клиент не записан	spaceInsteadOfFirstName
6	не дата, 31.02.2015, 29.02.2015; все поля с маской	birthDateIsNotDate, maskedFields
7	нет одного обязательного поля (17 вариантов) — ошибка	requiredFieldIsMissing
8	только обязательные поля — клиент записан	onlyRequiredFields
9	двойная проверка на клиенте и на сервере	листинги 4.2, 4.3

Поскольку web-клиент не пропускает некорректные данные до сервера, серверная валидация проверяется отдельно: интеграционные тесты ClientApiIT отправляют те же неверные данные прямо в REST, минуя браузер, и убеждаются, что сервис отвечает кодом 400 или 409 и ничего не пишет в базу. Пример теста каждого уровня приведён в листингах 5.1 и 5.2.

Листинг 5.1 – Интеграционный тест пункта 6 (ClientApiIT)

```
@ParameterizedTest(name = "дата рождения «{0}»")
@ValueSource(strings = {"не дата", "31.02.2015", "29.02.2015"})
@DisplayName("п.6 Текст, не являющийся датой, – ошибка")
void birthDateIsNotDate(String birthDate) throws Exception {
    create(TestClients.with("birthDate", birthDate))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields.birthDate").exists());

    assertThat(count("last_name = 'Иванов'")).isZero();
}

/** Проверка «прямо в базе»: SQL-запрос к таблице client в обход сервиса и репозиториев. */
private int count(String where) {
    return jdbc.queryForObject("select count(*) from client where " + where, Integer.class);
}
```

Листинг 5.2 – Функциональный тест пункта 2 через WebDriver (ClientUiIT)

```
@Test
@DisplayName("п.2 Другой клиент с паспортом студента – сообщение об ошибке")
void anotherClientWithSamePassport() {
    Map<String, Object> other = TestClients.valid();
    other.put("passportSeries", "MP");
    other.put("passportNumber", "3141592"); // паспорт клиента Лебедевич из начальных
    // данных
}
```

```

fill(other);
save();

assertThat(error("passportSeries")).isEqualTo("Клиент с таким паспортом уже
zaregistrirovann");
assertThat(count("passport_series = 'MP' and passport_number = '3141592'")).isEqualTo(1);
assertThat(count("last_name = 'Иванов'")).isZero();
}

```

5.3 Результаты прогона

Команда `./mvnw -pl client-service -am verify` собирает сервис и выполняет все 158 тестов; функциональные тесты запускают Chrome без окна, драйвер для него загружает Selenium Manager. Итог прогона приведён в листинге 5.3.

Листинг 5.3 – Результат выполнения тестов

```

$ ./mvnw -pl client-service -am verify

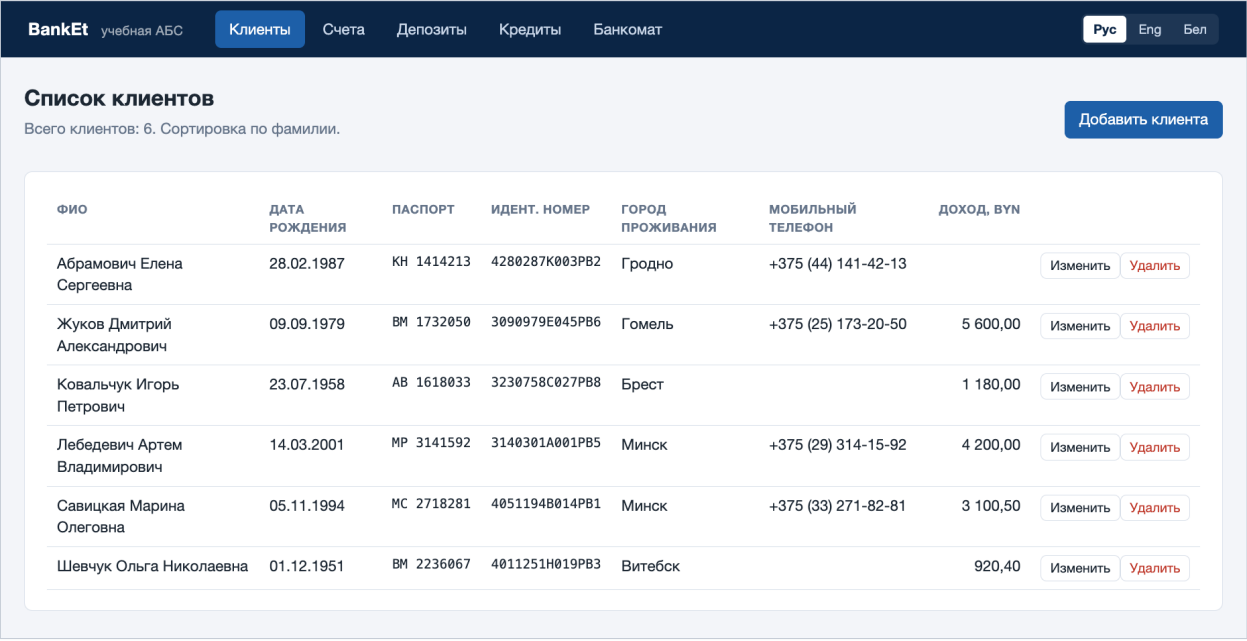
Tests run: 16, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.056 s -- in by.bsuir.bank.client.DateTextValidatorTest
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.585 s -- in by.bsuir.bank.client.ClientServiceTest
Tests run: 49, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.389 s -- in by.bsuir.bank.client.ClientRequestValidationTest
Tests run: 39, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.165 s -- in by.bsuir.bank.client.ClientApiIT
Tests run: 37, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 41.67 s -- in by.bsuir.bank.client.ClientUiIT
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.039 s -- in by.bsuir.bank.client.ClientLanguageIT

Tests run: 158, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

```

6 РЕЗУЛЬТАТЫ РАБОТЫ

После запуска сервис доступен по адресу `http://localhost:8081`. Форма «Список клиентов» с начальным заполнением базы показана на рисунке 6.1: шесть клиентов отсортированы по фамилии. В правой части шапки находится переключатель языка.



ФИО	ДАТА РОЖДЕНИЯ	ПАСПОРТ	ИДЕНТ. НОМЕР	ГОРОД ПРОЖИВАНИЯ	МОБИЛЬНЫЙ ТЕЛЕФОН	ДОХОД, BYN		
Абрамович Елена Сергеевна	28.02.1987	КН 1414213	4280287K003PB2	Гродно	+375 (44) 141-42-13		Изменить	Удалить
Жуков Дмитрий Александрович	09.09.1979	ВМ 1732050	3090979E045PB6	Гомель	+375 (25) 173-20-50	5 600,00	Изменить	Удалить
Ковальчук Игорь Петрович	23.07.1958	АВ 1618033	3230758C027PB8	Брест		1 180,00	Изменить	Удалить
Лебедев Артем Владимирович	14.03.2001	МР 3141592	3140301A001PB5	Минск	+375 (29) 314-15-92	4 200,00	Изменить	Удалить
Савицкая Марина Олеговна	05.11.1994	МС 2718281	4051194B014PB1	Минск	+375 (33) 271-82-81	3 100,50	Изменить	Удалить
Шевчук Ольга Николаевна	01.12.1951	ВМ 2236067	4011251H019PB3	Витебск		920,40	Изменить	Удалить

Рисунок 6.1 – Форма «Список клиентов»

Форма добавления клиента, заполненная корректными данными, приведена на рисунке 6.2. Обязательные поля отмечены звёздочкой, списки заполнены из справочников, значения полей с маской набраны с автоматической расстановкой разделителей.

BankEt учебная АБС
Клиенты
Счета
Депозиты
Кредиты
Банкомат

Рус
Eng
Бел

Новый клиент

Поля, отмеченные *, обязательны для заполнения.

ЛИЧНЫЕ ДАННЫЕ

Фамилия *
Иванов

Имя *
Иван

Отчество *
Иванович

Дата рождения *
17.05.1990

Пол *
☒ Мужской
☐ Женский

Место рождения *
г. Минск

ПАСПОРТНЫЕ ДАННЫЕ

Серия паспорта *
MP

№ паспорта *
7654321

Идентификационный номер *
3170590A077PB4

Кем выдан *
Фрунзенское РУВД г. Минска

Дата выдачи *
20.06.2018

ПРОЖИВАНИЕ И КОНТАКТЫ

Город фактического проживания *
Минск

Адрес фактического проживания *
ул. Притыцкого, д. 29, кв. 3

Город прописки *
Борисов

Телефон домашний
201-45-67

Телефон мобильный
+375 (29) 765-43-21

E-mail
ivanov@example.by

СОЦИАЛЬНЫЙ СТАТУС

Семейное положение *
Женат / замужем

Гражданство *
Республика Беларусь

Инвалидность *
Нет

Ежемесячный доход, BYN
2500.50

Пенсионер
☐ да

Сохранить

Отмена

Рисунок 6.2 – Форма добавления клиента

На рисунке 6.3 показана реакция первичной проверки на ошибки из программы защиты: цифры в фамилии, пробел вместо имени, дата 31.02.2015, неполный номер паспорта, мобильный телефон с недопустимым кодом и неполный e-mail. Запрос на сервер при этом не отправляется.

16

BankEtучебная АБС

КлиентыСчетаДепозитыКредитыБанкомат

РусEngБел

Новый клиент

Поля, отмеченные *, обязательны для заполнения.

Форма заполнена с ошибками — исправьте отмеченные поля

ЛИЧНЫЕ ДАННЫЕ

Фамилия *

1234

Допустимы только буквы

Имя *

Обязательное поле

Отчество *

Иванович

Дата рождения *

31.02.2015

Такой даты не существует, формат ДД.ММ.ГГГГ

Пол *

☒ Мужской☐ Женский

Место рождения *

г. Минск

ПАСПОРТНЫЕ ДАННЫЕ

Серия паспорта *

MP

№ паспорта *

12345

Номер паспорта — семь цифр

Идентификационный номер *

3170590A077PB4

Кем выдан *

Фрунзенское РУВД г. Минска

Дата выдачи *

20.06.2018

ПРОЖИВАНИЕ И КОНТАКТЫ

Город фактического проживания *

Минск

Адрес фактического проживания *

ул. Притыцкого, д. 29, кв. 3

Город прописки *

Борисов

Телефон домашний

201-45-67

Телефон мобильный

+375 (17) 765-43-21

Формат: +375 (29) 314-15-92, код 25, 29, 33 или 44

E-mail

ivanov@

Некорректный e-mail

СОЦИАЛЬНЫЙ СТАТУС

Семейное положение *

Женат / замужем

Гражданство *

Республика Беларусь

Инвалидность *

Нет

Ежемесячный доход, BYN

2500.50

Пенсионер

☐ да

Сохранить

Отмена

Рисунок 6.3 – Сообщения первичной проверки в браузере

Ошибку повторного ввода обнаруживает только сервер. На рисунке 6.4 новый клиент введён с паспортом и идентификационным номером, которые уже принадлежат клиенту из базы: сервис ответил кодом 409, а web-клиент показал его сообщения у соответствующих полей.

BankEtучебная АБС

КлиентыСчетаДепозитыКредитыБанкомат

РусEngБел

Новый клиент

Поля, отмеченные *, обязательны для заполнения.

Такой клиент уже есть в базе

ЛИЧНЫЕ ДАННЫЕ

Фамилия *Иванов

Имя *Иван

Отчество *Иванович

Дата рождения *17.05.1990

Пол *☒ Мужской ☐ Женский

Место рождения *г. Минск

ПАСПОРТНЫЕ ДАННЫЕ

Серия паспорта *MP

№ паспорта *3141592

Идентификационный номер *3140301A001PB5

Кем выдан *Фрунзенское РУВД г. Минска

Дата выдачи *20.06.2018

Клиент с таким паспортом уже зарегистрированКлиент с таким паспортом уже зарегистрированКлиент с таким идентификационным номером уже зарегистрирован

ПРОЖИВАНИЕ И КОНТАКТЫ

Город фактического проживания *Минск

Адрес фактического проживания *ул. Притыцкого, д. 29, кв. 3

Город прописки *Борисов

Телефон домашний201-45-67

Телефон мобильный+375 (29) 765-43-21

E-mailivanov@example.by

СОЦИАЛЬНЫЙ СТАТУС

Семейное положение *Женат / замужем

Гражданство *Республика Беларусь

Инвалидность *Нет

Ежемесячный доход, BYN2500.50

Пенсионер☐ да

Сохранить

Отмена

Рисунок 6.4 – Сообщения сервиса о нарушении уникальности

После исправления данных клиент сохраняется и появляется в списке на своём месте по алфавиту (рисунок 6.5).

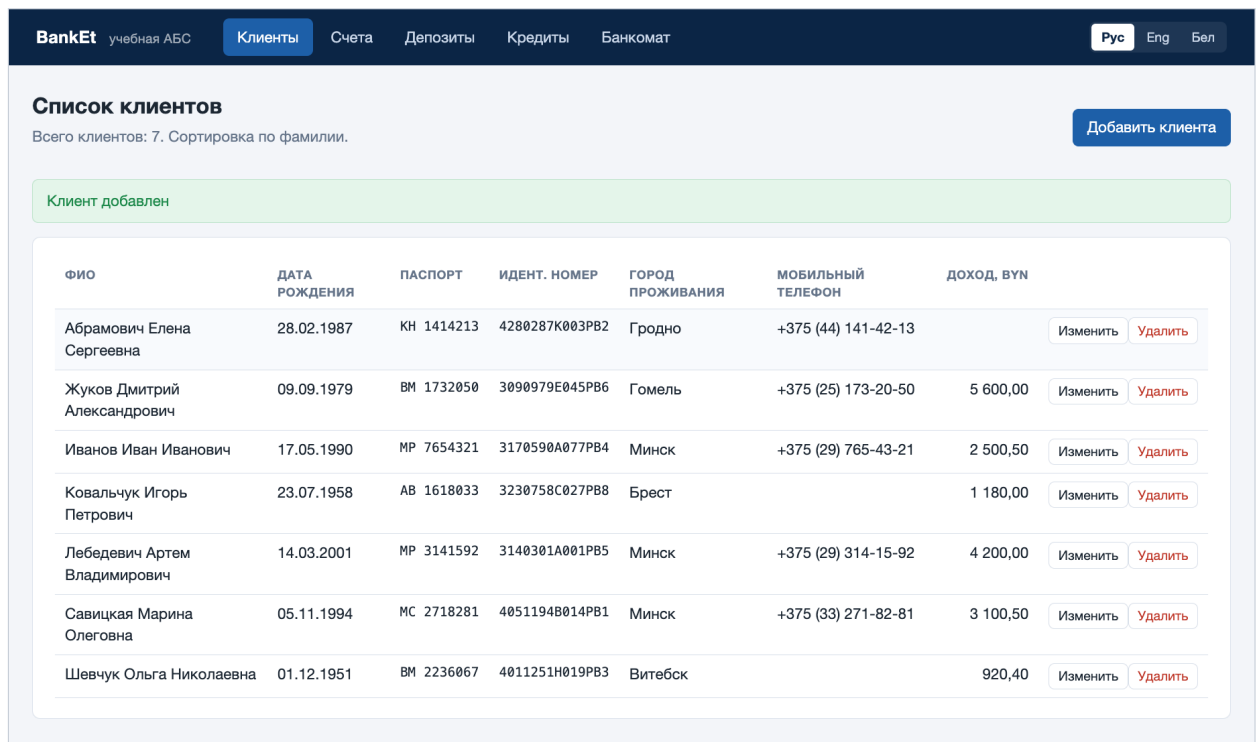


Рисунок 6.5 – Список после добавления клиента

Состояние таблицы можно проверить напрямую в консоли H2 по адресу /h2-console (JDBC URL jdbc:h2:mem:clients). На рисунке 6.6 видно, что после всех попыток в таблице семь записей и нет ни одинаковых паспортов, ни одинаковых идентификационных номеров, ни клиента с фамилией «1234».

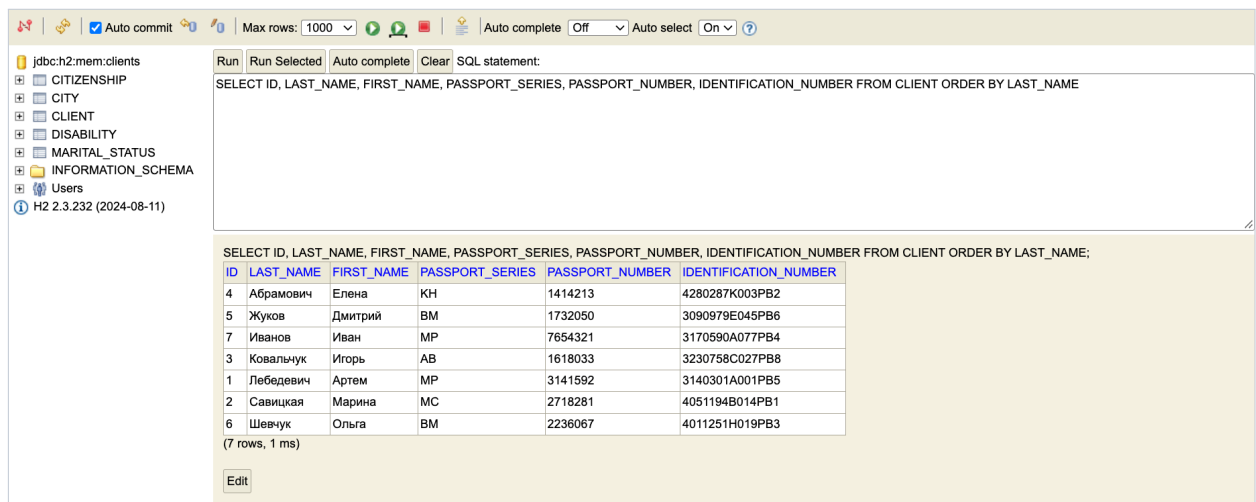


Рисунок 6.6 – Таблица client в консоли H2

Тот же интерфейс на других языках показан на рисунках 6.7 и 6.8. На английском выведен список клиентов: переведены заголовки, кнопки и название города из справочника, а ФИО остаются данными клиента. На белорусском показана форма с ошибками: сообщения первичной проверки взяты из словаря web-клиента.

BankEt

training core banking

Clients

Accounts

Deposits

Loans

ATM

Рус

Eng

Бел

Client list

Clients in total: 7. Sorted by last name.

Add client

FULL NAME	DATE OF BIRTH	PASSPORT	ID NUMBER	CITY OF RESIDENCE	MOBILE PHONE	INCOME, BYN	
Абрамович Елена Сергеевна	28.02.1987	КН 1414213	4280287K003PB2	Grodno	+375 (44) 141-42-13		EditDelete
Жуков Дмитрий Александрович	09.09.1979	ВМ 1732050	3090979E045PB6	Gomel	+375 (25) 173-20-50	5,600.00	EditDelete
Иванов Иван Иванович	17.05.1990	МР 7654321	3170590A077PB4	Minsk	+375 (29) 765-43-21	2,500.50	EditDelete
Ковальчук Игорь Петрович	23.07.1958	АВ 1618033	3230758C027PB8	Brest		1,180.00	EditDelete
Лебедев Артем Владимирович	14.03.2001	МР 3141592	3140301A001PB5	Minsk	+375 (29) 314-15-92	4,200.00	EditDelete
Савицкая Марина Олеговна	05.11.1994	МС 2718281	4051194B014PB1	Minsk	+375 (33) 271-82-81	3,100.50	EditDelete
Шевчук Ольга Николаевна	01.12.1951	ВМ 2236067	4011251H019PB3	Vitebsk		920.40	EditDelete

Рисунок 6.7 – Список клиентов на английском языке

BankEt вучэбная АБС							Кліенты		Рахункі	Дэпазіты	Крэдыты	Банкамат	Рус	Eng	Бел												
Новы кліент																											
Палі, пазначаныя *, абавязковыя для запаўнення.																											
Форма запоўнена з памылкамі — выпраўце пазначаныя палі																											
АСАБІСТЫЯ ДАНЫЯ																											
Прозвішча *				Імя *				Імя па бацьку *																			
1234				Іван				Івановіч																			
Дапушчальныя толькі літары																											
Дата нараджэння *				Пол *				Месца нараджэння *																			
31.02.2015				☒ Мужчынскі ☐ Жаночы				г. Мінск																			
Такой даты не існуе, фармат ДД.ММ.ГГГГ																											
ПАШПАРТНЫЯ ДАНЫЯ																											
Серыя пашпарта *				№ пашпарта *				Ідэнтыфікацыйны нумар *																			
МР				3141592				3140301A001PB5																			
Кім выдадзены *				Дата выдачы *																							
Фрунзенское РУВД г. Мінска				20.06.2018																							
ПРАЖЫВАННЕ І КАНТАКТЫ																											
Горад фактычнага пражывання *				Адрас фактычнага пражывання *																							
Мінск				ул. Притыцкого, д. 29, кв. 3																							

Рисунок 6.8 – Форма клиента на белорусском языке

ЗАКЛЮЧЕНИЕ

В ходе работы разработан микросервис «Клиенты» на Java и Spring Boot с базой данных H2 в памяти, REST-интерфейсом и web-клиентом на Vue 3. Схема данных содержит таблицу клиентов и четыре справочника для полей типа «Список»; база создаётся и заполняется автоматически при старте приложения.

Корректность данных обеспечивается на трёх уровнях. Web-клиент применяет маски ввода и проверяет форму до отправки, сервис повторяет проверку масок и обязательности средствами Bean Validation и контролирует уникальность паспорта и идентификационного номера, а база данных хранит ограничения уникальности и ссылочной целостности. Такое разделение соответствует практике реальных банковских систем: проверка в браузере служит удобству пользователя, а гарантии целостности даёт только сервер.

Интерфейс и сообщения сервиса доступны на русском, английском и белорусском языках: тексты web-клиента переводятся в браузере, сообщения сервиса и значения справочников — на сервере по заголовку `Accept-Language`.

Работоспособность подтверждена 158 автоматическими тестами: 73 модульными, 48 интеграционными и 37 функциональными, которые через WebDriver проходят все пункты программы защиты и сверяют результат с содержимым таблицы в базе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Уоллс, К. Spring в действии / К. Уоллс. — 6-е изд. — М.: ДМК Пресс, 2022. — 544 с.
2. Spring Boot Reference Documentation [Электронный ресурс]. — Режим доступа: <https://docs.spring.io/spring-boot/>. — Дата доступа: 01.10.2026.
3. Jakarta Bean Validation 3.0 Specification [Электронный ресурс]. — Режим доступа: <https://jakarta.ee/specifications/bean-validation/3.0/>. — Дата доступа: 01.10.2026.
4. Selenium WebDriver Documentation [Электронный ресурс]. — Режим доступа: <https://www.selenium.dev/documentation/webdriver/>. — Дата доступа: 01.10.2026.
5. Vue.js Guide [Электронный ресурс]. — Режим доступа: <https://vuejs.org/guide/>. — Дата доступа: 01.10.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код микросервиса «Клиенты»

Сборка проекта

Листинг А.1 – pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.16</version>
    <relativePath/>
  </parent>

  <groupId>by.bsuir.bank</groupId>
  <artifactId>bank</artifactId>
  <version>1.0.0</version>
  <packaging>pom</packaging>
  <name>bank</name>
  <description>Информационная система условного банка: микросервисы на Java + Spring</description>

  <modules>
    <module>ui-kit</module>
    <module>bank-common</module>
    <module>client-service</module>
    <module>account-service</module>
    <module>deposit-service</module>
    <module>credit-service</module>
    <module>atm-service</module>
  </modules>

  <properties>
    <java.version>17</java.version>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <vue.version>3.5.13</vue.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>by.bsuir.bank</groupId>
        <artifactId>ui-kit</artifactId>
        <version>${project.version}</version>
      </dependency>
      <dependency>
        <groupId>by.bsuir.bank</groupId>
        <artifactId>bank-common</artifactId>
        <version>${project.version}</version>
      </dependency>
      <dependency>
        <groupId>org.webjars.npm</groupId>
        <artifactId>vue</artifactId>
        <version>${vue.version}</version>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <build>
    <pluginManagement>
      <plugins>
        <!-- *Test – модульные тесты (фаза test), *IT – интеграционные и функциональные (фаза verify) -->
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-failsafe-plugin</artifactId>
          <executions>
            <execution>
```

```

        <goals>
            <goal>integration-test</goal>
            <goal>verify</goal>
        </goals>
    </execution>
</executions>
<configuration>
    <classesDirectory>${project.build.outputDirectory}</classesDirectory>
</configuration>
</plugin>
</plugins>
</pluginManagement>
</build>
</project>

```

Общие ресурсы web-клиентов (ui-kit)

Листинг A.2 – ui-kit/pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>by.bsuir.bank</groupId>
        <artifactId>bank</artifactId>
        <version>1.0.0</version>
    </parent>

    <artifactId>ui-kit</artifactId>
    <name>ui-kit</name>
    <description>Общие статические ресурсы web-клиентов: стили, маски ввода, обёртка над fetch</description>

    <dependencies>
        <!-- Vue 3 раздётся самим сервисом из webjar, без CDN и без сборки фронтенда -->
        <dependency>
            <groupId>org.webjars.npm</groupId>
            <artifactId>vue</artifactId>
        </dependency>
        <dependency>
            <groupId>org.webjars</groupId>
            <artifactId>webjars-locator-lite</artifactId>
        </dependency>
    </dependencies>
</project>

```

Листинг A.3 – ui-kit/src/main/resources/META-INF/resources/ui/bank.css

```

/* Общее оформление web-клиентов банка */
:root {
    --navy: #0b2545;
    --blue: #1d5fa8;
    --blue-soft: #e8f0fa;
    --ink: #1c2733;
    --muted: #66758a;
    --line: #d9e0ea;
    --bg: #f3f5f9;
    --ok: #1f8a4c;
    --ok-soft: #e4f5ea;
    --bad: #c0392b;
    --bad-soft: #fdecea;
    --radius: 8px;
}

* { box-sizing: border-box; }

body {
    margin: 0;
    background: var(--bg);
    color: var(--ink);
    font: 15px/1.45 -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif;
}

```

```

[v-cloak] { display: none; }

.topbar {
  display: flex;
  align-items: center;
  gap: 32px;
  padding: 0 28px;
  height: 56px;
  background: var(--navy);
  color: #fff;
}
.topbar .brand { font-weight: 700; font-size: 17px; letter-spacing: .2px; }
.topbar .brand span { font-weight: 400; opacity: .6; margin-left: 8px; font-size: 13px; }
.topbar nav { display: flex; gap: 4px; }
.topbar nav a {
  color: #c9d6e8;
  text-decoration: none;
  padding: 8px 14px;
  border-radius: 6px;
}
.topbar nav a:hover { background: rgba(255, 255, 255, .08); color: #fff; }
.topbar nav a.active { background: var(--blue); color: #fff; }
.topbar .languages { margin-left: auto; display: flex; gap: 2px; background: rgba(255, 255, 255, .08); border-radius: 6px; padding: 2px; }
.topbar .languages button { background: none; border: none; color: #c9d6e8; padding: 4px 10px; font-size: 13px; border-radius: 4px; }
.topbar .languages button:hover { color: #fff; }
.topbar .languages button.active { background: #fff; color: var(--navy); font-weight: 600; }

main { max-width: 1240px; margin: 24px auto 48px; padding: 0 24px; }

h1 { font-size: 22px; margin: 0 0 4px; }
h2 { font-size: 17px; margin: 0 0 12px; }
.subtitle { color: var(--muted); margin: 0 0 18px; }

.card {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: var(--radius);
  padding: 20px 22px;
  margin-bottom: 18px;
}

.toolbar { display: flex; align-items: center; justify-content: space-between; gap: 12px; margin-bottom: 14px; }
.toolbar .spacer { flex: 1; }

button, .btn {
  font: inherit;
  cursor: pointer;
  border: 1px solid var(--line);
  background: #fff;
  color: var(--ink);
  padding: 7px 14px;
  border-radius: 6px;
}
button:hover { border-color: var(--blue); }
button.primary { background: var(--blue); border-color: var(--blue); color: #fff; }
button.primary:hover { background: #174f8d; }
button.danger { color: var(--bad); }
button.danger:hover { border-color: var(--bad); background: var(--bad-soft); }
button.small { padding: 3px 9px; font-size: 13px; }
button.disabled { opacity: .5; cursor: default; }

table { width: 100%; border-collapse: collapse; }
th, td { text-align: left; padding: 8px 10px; border-bottom: 1px solid var(--line); vertical-align: top; }
th { font-size: 12px; text-transform: uppercase; letter-spacing: .4px; color: var(--muted); font-weight: 600; }
tbody tr:hover { background: #f8fafd; }
td.num, th.num { text-align: right; font-variant-numeric: tabular-nums; white-space: nowrap; }
td.mono { font-family: "SF Mono", Menlo, Consolas, monospace; font-size: 13px; white-space: nowrap; }
td.actions { white-space: nowrap; text-align: right; }
tfoot td { font-weight: 600; border-bottom: none; }
.empty { color: var(--muted); padding: 18px 0; text-align: center; }

.form-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 14px 20px; }
.form-grid .wide { grid-column: span 2; }
.form-grid .full { grid-column: 1 / -1; }
fieldset { border: 1px solid var(--line); border-radius: var(--radius); margin: 0 0 16px; padding: 14px 18px 18px; }
legend { padding: 0 8px; color: var(--muted); font-size: 13px; text-transform: uppercase; letter-spacing: .4px; }

.field label { display: block; font-size: 13px; color: var(--muted); margin-bottom: 4px; }
.field label.req { color: var(--bad); }
.field input[type=text], .field select {

```

```

width: 100%;
font: inherit;
padding: 7px 10px;
border: 1px solid var(--line);
border-radius: 6px;
background: #fff;
}
.field input:focus, .field select:focus { outline: 2px solid var(--blue-soft); border-color: var(--blue); }
.field input[readonly] { background: #f3f5f9; color: var(--muted); }
.field.invalid input, .field.invalid select { border-color: var(--bad); background: var(--bad-soft); }
.field .error { color: var(--bad); font-size: 12.5px; margin-top: 3px; }
.field .hint { color: var(--muted); font-size: 12.5px; margin-top: 3px; }
.field .choices { display: flex; gap: 18px; padding: 7px 0; }
.field .choices label { display: inline-flex; align-items: center; gap: 6px; color: var(--ink); font-size: 15px; margin: 0; }

.form-actions { display: flex; gap: 10px; margin-top: 6px; }

.alert { padding: 10px 14px; border-radius: 6px; margin-bottom: 14px; }
.alert.error { background: var(--bad-soft); color: var(--bad); border: 1px solid #f3c2bc; }
.alert.success { background: var(--ok-soft); color: var(--ok); border: 1px solid #b9e2c7; }
.alert.info { background: var(--blue-soft); color: var(--navy); border: 1px solid #c5d8ef; }

.badge { display: inline-block; padding: 1px 8px; border-radius: 10px; font-size: 12px; background: var(--blue-soft); color: var(--navy); }
.badge.ok { background: var(--ok-soft); color: var(--ok); }
.badge.off { background: #eceff3; color: var(--muted); }
.badge.bad { background: var(--bad-soft); color: var(--bad); }

.stats { display: grid; grid-template-columns: repeat(4, 1fr); gap: 14px; margin-bottom: 18px; }
.stat { background: #fff; border: 1px solid var(--line); border-radius: var(--radius); padding: 14px 18px; }
.stat .label { color: var(--muted); font-size: 12px; text-transform: uppercase; letter-spacing: .4px; }
.stat .value { font-size: 20px; font-weight: 600; margin-top: 2px; font-variant-numeric: tabular-nums; }

.muted { color: var(--muted); }
.nowrap { white-space: nowrap; }
.right { text-align: right; }

.bankday { display: flex; align-items: center; gap: 8px; background: #fff; border: 1px solid var(--line); border-radius: var(--radius); padding: 8px 12px; }
.bankday .label { color: var(--muted); font-size: 12px; text-transform: uppercase; letter-spacing: .4px; }
.bankday b { font-size: 16px; margin-right: 8px; font-variant-numeric: tabular-nums; }
.bankday input { width: 56px; font: inherit; padding: 6px 8px; border: 1px solid var(--line); border-radius: 6px; text-align: right; }

.details { display: grid; grid-template-columns: repeat(4, 1fr); gap: 12px 20px; }
.details .label { color: var(--muted); font-size: 12px; text-transform: uppercase; letter-spacing: .4px; }
.details .value { font-weight: 600; }
a.link { color: var(--blue); cursor: pointer; text-decoration: none; }
a.link:hover { text-decoration: underline; }

```

Листинг А.4 – ui-kit/src/main/resources/META-INF/resources/ui/bank.js

```

/* Общие помощники web-клиентов банка: язык интерфейса, REST-вызовы, маски ввода, форматирование. */
const Bank = {

  /* Адреса web-клиентов остальных микросервисов для общей навигации. */
  services: [
    { key: 'clients', port: 8081 },
    { key: 'accounts', port: 8082 },
    { key: 'deposits', port: 8083 },
    { key: 'credits', port: 8084 },
    { key: 'atm', port: 8085 },
  ],

  /* ----- язык интерфейса ----- */

  languages: [
    { code: 'ru', title: 'Рус', locale: 'ru-RU' },
    { code: 'en', title: 'Eng', locale: 'en-GB' },
    { code: 'be', title: 'Бел', locale: 'be-BY' },
  ],

  /* Выбор хранится в cookie: она общая для всех портов localhost, поэтому язык один для всех пяти web-клиентов. */
  lang: (document.cookie.match(/(?:^|;)bank_lang=(ru|en|be)/) || [])[1] || 'ru',

  /* Словари: общая часть – в ui-kit/i18n.js, тексты приложения – в его собственном i18n.js. */
  messages: { ru: {}, en: {}, be: {} },

  addMessages(dictionary) {
    for (const lang of Object.keys(dictionary)) Object.assign(Bank.messages[lang], dictionary[lang]);
  }
}

```

```

},

/* Текст по ключу на выбранном языке; {0}, {1} заменяются аргументами. Непереведённый ключ берётся из русского словаря. */
t(key, ...args) {
  const text = Bank.messages[Bank.lang][key] ?? Bank.messages.ru[key] ?? key;
  return text.replace(/\{(\d+)\}/g, (match, index) => args[index] ?? '');
},

setLang(code) {
  document.cookie = 'bank_lang=' + code + '; path=/; max-age=31536000; SameSite=Lax';
  location.reload();
},

locale() {
  return Bank.languages.find(language => language.code === Bank.lang).locale;
},

/* ----- REST ----- */

/* REST-вызов: язык передаётся сервису заголовком Accept-Language; при ошибке бросает { status, message, fields }. */
async api(method, url, body) {
  const options = { method, headers: { 'Accept': 'application/json', 'Accept-Language': Bank.lang } };
  if (body !== undefined) {
    options.headers['Content-Type'] = 'application/json';
    options.body = JSON.stringify(body);
  }
  let response;
  try {
    response = await fetch(url, options);
  } catch (e) {
    throw { status: 0, message: Bank.t('common.unavailable'), fields: {} };
  }
  const text = await response.text();
  const data = text ? JSON.parse(text) : null;
  if (!response.ok) {
    throw {
      status: response.status,
      message: (data && data.message) || Bank.t('common.error', response.status),
      fields: (data && data.fields) || {},
    };
  }
  return data;
},

/* ----- ввод и форматирование ----- */

/* Маска ввода: # — цифра, A — латинская буква (в верхнем регистре), остальное — литералы. */
mask(value, pattern) {
  const raw = String(value ?? '');
  let out = '';
  let i = 0;
  for (const p of pattern) {
    if (i >= raw.length) break;
    if (p === '#' || p === 'A') {
      const allowed = p === '#' ? /\d/ : /[a-zA-Z]/;
      while (i < raw.length && !allowed.test(raw[i])) i++;
      if (i >= raw.length) break;
      out += raw[i++].toUpperCase();
    } else {
      out += p;
      if (raw[i] === p) i++;
    }
  }
  return out;
},

/* Директива v-mask="'###.###.####'" — применяет маску при каждом вводе. */
maskDirective: {
  mounted(el, binding) {
    el.addEventListener('input', () => {
      const masked = Bank.mask(el.value, binding.value);
      if (masked !== el.value) {
        el.value = masked;
        el.dispatchEvent(new Event('input'));
      }
    });
  },
},

/* Строгий разбор даты ДД.ММ.ГГГГ: 31.02.2015 и 29.02.2015 датами не считаются. */
parseDate(text) {
  const m = /^(\d{2})\.(\d{2})\.(\d{4})$/ .exec(text || '');

```

```

    if (!m) return null;
    const [day, month, year] = [Number(m[1]), Number(m[2]), Number(m[3])];
    const date = new Date(year, month - 1, day);
    const same = date.getFullYear() === year && date.getMonth() === month - 1 && date.getDate() === day;
    return same ? date : null;
  },

  money(value) {
    if (value === null || value === undefined || value === '') return '';
    return Number(value).toLocaleString(Bank.locale(), { minimumFractionDigits: 2, maximumFractionDigits: 2 });
  },

  /* ISO-дата 2026-10-01 -> 01.10.2026 */
  date(iso) {
    return iso ? String(iso).split('-').reverse().join('.') : '';
  },

  /* ----- общие компоненты ----- */

  /* Создание приложения Vue: функция t доступна в шаблонах, подключены шапка, панель дня и маски. */
  createApp(options, pageTitleKey) {
    document.documentElement.lang = Bank.lang;
    document.title = Bank.t(pageTitleKey) + ' - BankEt';
    const app = Vue.createApp(options);
    app.config.globalProperties.t = Bank.t;
    app.component('bank-nav', Bank.navComponent);
    app.component('bank-day', Bank.dayComponent);
    app.directive('mask', Bank.maskDirective);
    return app;
  },

  /* Панель банковского дня: текущая дата и процедура «Закрытие банковского дня». */
  dayComponent: {
    emits: ['changed', 'failed'],
    data() {
      return { date: null, days: 30, busy: false };
    },
    async mounted() {
      try {
        this.date = (await Bank.api('GET', '/api/bank-day')).date;
      } catch (e) {
        this.$emit('failed', e);
      }
    },
    methods: {
      async close(days) {
        this.busy = true;
        try {
          const result = await Bank.api('POST', '/api/bank-day/close?days=' + days);
          this.date = result.bankDate;
          this.$emit('changed', result);
        } catch (e) {
          this.$emit('failed', e);
        } finally {
          this.busy = false;
        }
      },
      format: iso => Bank.date(iso),
    },
    template: `
      <div class="bankday">
        <span class="label">{{ t('day.label') }}</span>
        <b id="bank-date">{{ format(date) }}</b>
        <button id="btn-close-day" class="primary" :disabled="busy" @click="close(1)">{{ t('day.close') }}</button>
        <input id="days" type="text" v-model.number="days" maxlength="4">
        <button id="btn-close-days" :disabled="busy || !(days > 0)" @click="close(days)">{{ t('day.closeDays') }}</button>
      </div>`,
  },

  /* Шапка: переходы между web-клиентами микросервисов и выбор языка. */
  navComponent: {
    props: ['active'],
    data() {
      return {
        services: Bank.services,
        languages: Bank.languages,
        lang: Bank.lang,
        host: location.protocol + '//' + location.hostname,
      };
    },
    methods: {
      setLang: code => Bank.setLang(code),
    },
  },

```

```

    },
    template: `
      <header class="topbar">
        <div class="brand">BankEt <span>{{ t('brand.subtitle') }}</span></div>
        <nav>
          <a v-for="s in services" :key="s.key" :href="host + ':' + s.port + '/'"
            :class="{ active: s.key === active }">{{ t('nav.' + s.key) }}</a>
        </nav>
        <div class="languages">
          <button v-for="l in languages" :key="l.code" :id="'lang-' + l.code"
            :class="{ active: l.code === lang }" @click="setLang(l.code)">{{ l.title }}</button>
        </div>
      </header>`,
  },
};

```

Листинг A.5 – ui-kit/src/main/resources/META-INF/resources/ui/i18n.js

```

/* Общий словарь web-клиентов: шапка, панель банковского дня, таблицы счетов и проводок, формы договоров. */
Bank.addMessages({
  ru: {
    'brand.subtitle': 'учебная АБС',
    'nav.clients': 'Клиенты', 'nav.accounts': 'Счета', 'nav.deposits': 'Депозиты', 'nav.credits': 'Кредиты', 'nav.atm': 'Банкомат',
    'day.label': 'Банковский день', 'day.close': 'Закрыть день', 'day.closeDays': 'Закрыть дней',
    'common.unavailable': 'Сервис недоступен',
    'common.error': 'Ошибка {0}',
    'common.required': 'Обязательное поле',
    'common.formErrors': 'Форма заполнена с ошибками — исправьте отмеченные поля',
    'common.cancel': 'Отмена',
    'common.toList': 'К списку',
    'common.total': 'Итого, {0}',
    'events.title': 'Протокол закрытия дня',
    'events.more': '... всего записей: {0}',
    'table.account': 'Номер счёта', 'table.code': 'Код', 'table.purpose': 'Назначение', 'table.activity': 'Активность',
    'table.debit': 'Дебет', 'table.credit': 'Кредит', 'table.balance': 'Сальдо', 'table.date': 'Дата',
    'table.operation': 'Операция', 'table.accountShort': 'Счёт', 'table.contract': 'Договор',
    'contract.accounts': 'Счета договора',
    'contract.postings': 'Проводки по договору',
    'contract.count': 'Договоров: {0}',
    'contract.conclude': 'Заклчить договор',
    'contract.none': 'Договоров пока нет',
    'contract.client': 'Клиент',
    'contract.chooseClient': '— выберите клиента —',
    'contract.passport': 'паспорт',
    'contract.number': 'Номер договора',
    'contract.currency': 'Валюта',
    'contract.chooseProduct': '— выберите программу —',
    'contract.term': 'Срок договора, мес.',
    'contract.startDate': 'Дата начала',
    'contract.endDate': 'Дата окончания',
    'contract.perAnnum': '% годовых',
    'contract.months': 'мес.',
    'contract.sum': 'Сумма', 'contract.rate': 'Ставка', 'contract.start': 'Начало', 'contract.end': 'Окончание',
    'contract.status': 'Статус', 'contract.period': 'Период', 'contract.termShort': 'Срок',
    'contract.active': 'Действует',
    'contract.moneyFormat': 'Денежная сумма: только цифры и не более 2 знаков после запятой',
    'contract.termInteger': 'Срок — целое число месяцев',
    'contract.termRange': 'Срок по программе: от {0} до {1} мес.',
  },
  en: {
    'brand.subtitle': 'training core banking',
    'nav.clients': 'Clients', 'nav.accounts': 'Accounts', 'nav.deposits': 'Deposits', 'nav.credits': 'Loans', 'nav.atm': 'ATM',
    'day.label': 'Banking day', 'day.close': 'Close day', 'day.closeDays': 'Close days',
    'common.unavailable': 'Service unavailable',
    'common.error': 'Error {0}',
    'common.required': 'Required field',
    'common.formErrors': 'The form contains errors — please correct the highlighted fields',
    'common.cancel': 'Cancel',
    'common.toList': 'Back to list',
    'common.total': 'Total, {0}',
    'events.title': 'Day closing log',
    'events.more': '... entries in total: {0}',
    'table.account': 'Account number', 'table.code': 'Code', 'table.purpose': 'Purpose', 'table.activity': 'Type',
    'table.debit': 'Debit', 'table.credit': 'Credit', 'table.balance': 'Balance', 'table.date': 'Date',
    'table.operation': 'Operation', 'table.accountShort': 'Account', 'table.contract': 'Contract',
    'contract.accounts': 'Contract accounts',
    'contract.postings': 'Contract postings',
    'contract.count': 'Contracts: {0}',
    'contract.conclude': 'Sign contract',
  }
});

```

```

'contract.none': 'No contracts yet',
'contract.client': 'Client',
'contract.chooseClient': '— select a client —',
'contract.passport': 'passport',
'contract.number': 'Contract number',
'contract.currency': 'Currency',
'contract.chooseProduct': '— select a programme —',
'contract.term': 'Contract term, months',
'contract.startDate': 'Start date',
'contract.endDate': 'End date',
'contract.perAnnum': '% per annum',
'contract.months': 'мо.',
'contract.sum': 'Amount', 'contract.rate': 'Rate', 'contract.start': 'Start', 'contract.end': 'End',
'contract.status': 'Status', 'contract.period': 'Period', 'contract.termShort': 'Term',
'contract.active': 'Active',
'contract.moneyFormat': 'Amount: digits only, no more than 2 decimal places',
'contract.termInteger': 'The term is a whole number of months',
'contract.termRange': 'Programme term: from {0} to {1} months',
},
be: {
'brand.subtitle': 'вучэбная АБС',
'nav.clients': 'Кліенты', 'nav.accounts': 'Рахункі', 'nav.deposits': 'Дэпазіты', 'nav.credits': 'Крэдыты', 'nav.atm': 'Банкамат',
'day.label': 'Банкаўскі дзень', 'day.close': 'Закрыць дзень', 'day.closeDays': 'Закрыць дзён',
'common.unavailable': 'Сэрвіс недаступны',
'common.error': 'Памылка {0}',
'common.required': 'Абавязковае поле',
'common.formErrors': 'Форма запоўнена з памылкамі — выпраўце пазначаныя палі',
'common.cancel': 'Адмена',
'common.toList': 'Да спіса',
'common.total': 'Разам, {0}',
'events.title': 'Прадакол закрыцця дня',
'events.more': '... усяго запісаў: {0}',
'table.account': 'Нумар рахунку', 'table.code': 'Код', 'table.purpose': 'Прызначэнне', 'table.activity': 'Актыўнасць',
'table.debit': 'Дэбет', 'table.credit': 'Крэдыт', 'table.balance': 'Сальда', 'table.date': 'Дата',
'table.operation': 'Аперацыя', 'table.accountShort': 'Рахунак', 'table.contract': 'Дагавор',
'contract.accounts': 'Рахункі дагавора',
'contract.postings': 'Праводкі па дагаворы',
'contract.count': 'Дагавораў: {0}',
'contract.conclude': 'Заклучыць дагавор',
'contract.none': 'Дагавораў пакуль няма',
'contract.client': 'Кліент',
'contract.chooseClient': '— выберыце кліента —',
'contract.passport': 'пашпарт',
'contract.number': 'Нумар дагавора',
'contract.currency': 'Валюта',
'contract.chooseProduct': '— выберыце праграму —',
'contract.term': 'Тэрмін дагавора, мес.',
'contract.startDate': 'Дата пачатку',
'contract.endDate': 'Дата заканчэння',
'contract.perAnnum': '% гадавых',
'contract.months': 'мес.',
'contract.sum': 'Сума', 'contract.rate': 'Стаўка', 'contract.start': 'Пачатак', 'contract.end': 'Заканчэнне',
'contract.status': 'Статус', 'contract.period': 'Перыяд', 'contract.termShort': 'Тэрмін',
'contract.active': 'Дзейнічае',
'contract.moneyFormat': 'Грашовая сума: толькі лічбы і не больш за 2 знакі пасля коскі',
'contract.termInteger': 'Тэрмін — цэлы лік месяцаў',
'contract.termRange': 'Тэрмін па праграме: ад {0} да {1} мес.',
},
});

```

Локализация (bank-common, пакет i18n)

Листинг А.6 — bank-common/src/main/resources/common-messages.properties

```

# Общие тексты сервисов — русский язык (по умолчанию)
validation.required=Обязательное поле
error.checkFields=Проверьте правильность заполнения полей
error.badRequest=Некорректный формат запроса
error.downstream=Ошибка смежного сервиса
error.unavailable=Смежный сервис недоступен, повторите операцию позже

# Наименования собственных счетов банка
account.cash=Касса банка, {0}
account.fund=Фонд развития банка, {0}
account.operator=Расчётный счёт оператора {0}

```

```

# Проводки: главная книга хранит ключ, текст подставляется при чтении
posting.fund.capital=Формирование фонда развития банка
posting.deposit.cashIn=Внесение денег в кассу
posting.deposit.toAccount=Перевод денег с кассы на текущий счёт
posting.deposit.toFund=Использование денег банком
posting.deposit.accrual=Начисление процентов по депозиту
posting.deposit.interestToCash=Перевод процентов в кассу
posting.deposit.interestOut=Вывод процентов из кассы
posting.deposit.maturity=Окончание депозита
posting.deposit.toCash=Перевод депозита в кассу
posting.deposit.cashOut=Вывод денег из кассы
posting.credit.issue=Выделение кредита банком
posting.credit.toCash=Перевод кредита в кассу
posting.credit.cashOut=Получение кредита через кассу
posting.credit.interestIn=Внесение процентов в кассу
posting.credit.interestFromCash=Перевод процентов из кассы
posting.credit.interestCharge=Начисление процентов банком
posting.credit.cashIn=Внесение денег в кассу
posting.credit.repay=Погашение долга за кредит из кассы
posting.credit.partReturn=Возврат части кредита в фонд банка
posting.credit.maturity=Окончание кредита
posting.atm.payment=Оплата услуг связи {0}, номер {1}

# Проверки договоров – общие для депозитов и кредитов
validation.amountPositive=Сумма должна быть больше нуля
validation.amountFormat=Денежная сумма: не более 2 знаков после запятой
validation.termPositive=Срок должен быть больше нуля
contract.duplicate=Договор уже существует
contract.duplicateNumber=Договор с таким номером уже заключён
contract.productCurrency=Программа недоступна в валюте {0}
contract.rate=Ставка по программе – {0} %
contract.termRange=Срок по программе: от {0} до {1} мес.
contract.startDate=Договор заключается текущим банковским днём: {0}
contract.endDate=Дата окончания не соответствует сроку договора
contract.clientNotFound=Клиент не найден в модуле «Клиенты»

```

Листинг А.7 – bank-common/src/main/resources/common-messages_be.properties

```

# Агульныя тэксты сэрвісаў – беларуская мова
validation.required=Абавязковае поле
error.checkFields=Праверце правільнасць запаўнення палёў
error.badRequest=Некарэктны фармат запиту
error.downstream=Памылка сумежнага сэрвісу
error.unavailable=Сумежны сэрвіс недаступны, паўтарыце аперацыю пазней

account.cash=Каса банка, {0}
account.fund=Фонд развіцця банка, {0}
account.operator=Разліковы рахунак аператара {0}

posting.fund.capital=Фарміраванне фонду развіцця банка
posting.deposit.cashIn=Унясенне грошай у касу
posting.deposit.toAccount=Перавод грошай з касы на бягучы рахунак
posting.deposit.toFund=Выкарыстанне грошай банкам
posting.deposit.accrual=Налічэнне працэнтаў па дэпазіце
posting.deposit.interestToCash=Перавод працэнтаў у касу
posting.deposit.interestOut=Вывад працэнтаў з касы
posting.deposit.maturity=Заканчэнне дэпазіту
posting.deposit.toCash=Перавод дэпазіту ў касу
posting.deposit.cashOut=Вывад грошай з касы
posting.credit.issue=Выдзяленне крэдыту банкам
posting.credit.toCash=Перавод крэдыту ў касу
posting.credit.cashOut=Атрыманне крэдыту праз касу
posting.credit.interestIn=Унясенне працэнтаў у касу
posting.credit.interestFromCash=Перавод працэнтаў з касы
posting.credit.interestCharge=Налічэнне працэнтаў банкам
posting.credit.cashIn=Унясенне грошай у касу
posting.credit.repay=Пагашэнне доўгу па крэдыце з касы
posting.credit.partReturn=Вяртанне часткі крэдыту ў фонд банка
posting.credit.maturity=Заканчэнне крэдыту
posting.atm.payment=Аплата паслуг сувязі {0}, нумар {1}

validation.amountPositive=Сума павінна быць большай за нуль
validation.amountFormat=Грашовая сума: не больш за 2 знакі пасля коскі
validation.termPositive=Тэрмін павінен быць большым за нуль
contract.duplicate=Дагавор ужо існуе
contract.duplicateNumber=Дагавор з такім нумарам ужо заключаны
contract.productCurrency=Праграма недаступная ў валюце {0}
contract.rate=Стаўка па праграме – {0} %
contract.termRange=Тэрмін па праграме: ад {0} да {1} мес.

```

```
contract.startDate=Дагавор заключаецца бягучым банкаўскім днём: {0}
contract.endDate=Дата заканчэння не адпавядае тэрміну дагавора
contract.clientNotFound=Кліент не знойдзены ў модулі «Кліенты»
```

Листинг А.8 –

bank-common/src/main/resources/common-messages_en.properties

```
# Common service texts – English
validation.required=Required field
error.checkFields=Please check the highlighted fields
error.badRequest=Malformed request
error.downstream=Adjacent service error
error.unavailable=An adjacent service is unavailable, please try again later

account.cash=Bank cash desk, {0}
account.fund=Bank development fund, {0}
account.operator=Settlement account of operator {0}

posting.fund.capital=Bank development fund formed
posting.deposit.cashIn=Cash paid in at the cash desk
posting.deposit.toAccount=Transfer from the cash desk to the current account
posting.deposit.toFund=Funds used by the bank
posting.deposit.accrual=Deposit interest accrued
posting.deposit.interestToCash=Interest transferred to the cash desk
posting.deposit.interestOut=Interest paid out from the cash desk
posting.deposit.maturity=Deposit matured
posting.deposit.toCash=Deposit transferred to the cash desk
posting.deposit.cashOut=Cash paid out from the cash desk
posting.credit.issue=Loan allocated by the bank
posting.credit.toCash=Loan transferred to the cash desk
posting.credit.cashOut=Loan received at the cash desk
posting.credit.interestIn=Interest paid in at the cash desk
posting.credit.interestFromCash=Interest transferred from the cash desk
posting.credit.interestCharge=Interest charged by the bank
posting.credit.cashIn=Cash paid in at the cash desk
posting.credit.repay=Loan repayment from the cash desk
posting.credit.partReturn=Part of the loan returned to the bank fund
posting.credit.maturity=Loan closed
posting.atm.payment=Mobile service payment {0}, number {1}

validation.amountPositive=The amount must be greater than zero
validation.amountFormat=Amount: no more than 2 decimal places
validation.termPositive=The term must be greater than zero
contract.duplicate=The contract already exists
contract.duplicateNumber=A contract with this number has already been signed
contract.productCurrency=The programme is not available in {0}
contract.rate=Programme rate: {0} %
contract.termRange=Programme term: from {0} to {1} months
contract.startDate=The contract is signed on the current banking day: {0}
contract.endDate=The end date does not match the contract term
contract.clientNotFound=The client was not found in the Clients module
```

Листинг А.9 –

bank-common/src/main/java/by/bsuir/bank/common/i18n/I18nConfig.java

```
package by.bsuir.bank.common.i18n;

import org.springframework.context.MessageSource;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.LocaleResolver;
import org.springframework.web.servlet.i18n.AcceptHeaderLocaleResolver;

/**
 * Язык запроса определяется заголовком Accept-Language: web-клиент подставляет в него язык,
 * выбранный пользователем, а сервисы передают его дальше при обращении друг к другу.
 */
@Configuration
public class I18nConfig {

    /** Тот же источник текстов используется Bean Validation для сообщений вида {ключ}. */
    @Bean
    public MessageSource messageSource() {
        return Messages.source();
    }

    @Bean
    public LocaleResolver localeResolver() {

```

```

        AcceptHeaderLocaleResolver resolver = new AcceptHeaderLocaleResolver();
        resolver.setSupportedLocales(Messages.SUPPORTED);
        resolver.setDefaultLocale(Messages.DEFAULT);
        return resolver;
    }
}

```

Листинг А.10 – bank-common/src/main/java/by/bsuir/bank/common/i18n/Localized.java

```

package by.bsuir.bank.common.i18n;

import java.util.Arrays;

/**
 * Тексты, которые система сама записывает в базу (названия проводов, наименования счетов банка),
 * хранятся не на конкретном языке, а кодом вида «@ключ|аргумент|аргумент» и превращаются
 * в текст при чтении – на языке того, кто читает.
 */
public final class Localized {

    private static final String MARK = "@";
    private static final String SEPARATOR = "|";

    public static String code(String key, Object... args) {
        StringBuilder code = new StringBuilder(MARK).append(key);
        for (Object arg : args) {
            code.append(SEPARATOR).append(arg);
        }
        return code.toString();
    }

    /** Обычный текст (например, ФИО клиента в названии счёта) возвращается как есть. */
    public static String render(String text) {
        if (text == null || !text.startsWith(MARK)) {
            return text;
        }
        String[] parts = text.substring(MARK.length()).split("\\\\" + SEPARATOR);
        return Messages.get(parts[0], (Object[]) Arrays.copyOfRange(parts, 1, parts.length));
    }

    private Localized() {
    }
}

```

Листинг А.11 – bank-common/src/main/java/by/bsuir/bank/common/i18n/Messages.java

```

package by.bsuir.bank.common.i18n;

import org.springframework.context.MessageSource;
import org.springframework.context.NoSuchMessageException;
import org.springframework.context.i18n.LocaleContextHolder;
import org.springframework.context.support.ResourceBundleMessageSource;

import java.util.Arrays;
import java.util.List;
import java.util.Locale;

/**
 * Тексты на языке пользователя: русском, английском или белорусском.
 * Словари – messages*.properties сервиса и common-messages*.properties общей библиотеки;
 * файл без суффикса содержит русские тексты и служит языком по умолчанию.
 */
public final class Messages {

    public static final Locale RU = Locale.forLanguageTag("ru");
    public static final Locale EN = Locale.forLanguageTag("en");
    public static final Locale BE = Locale.forLanguageTag("be");
    public static final Locale DEFAULT = RU;
    public static final List<Locale> SUPPORTED = List.of(RU, EN, BE);

    private static final ResourceBundleMessageSource SOURCE = new ResourceBundleMessageSource();

    static {
        SOURCE.setBasenames("messages", "common-messages");
        SOURCE.setDefaultEncoding("UTF-8");
        SOURCE.setFallbackToSystemLocale(false);
    }
}

```

```

        // вне HTTP-запроса (старт приложения, фоновые задачи) используется русский язык
        LocaleContextHolder.setDefaultLocale(DEFAULT);
    }

    public static MessageSource source() {
        return SOURCE;
    }

    /** Язык текущего запроса; неподдерживаемый язык заменяется русским. */
    public static Locale locale() {
        String language = LocaleContextHolder.getLocale().getLanguage();
        return SUPPORTED.stream().filter(locale -> locale.getLanguage().equals(language)).findFirst().orElse(DEFAULT);
    }

    /** Текст по ключу. Аргументы подставляются как строки, чтобы числа и суммы не переформатировались. */
    public static String get(String code, Object... args) {
        Object[] strings = Arrays.stream(args).map(String::valueOf).toArray();
        try {
            return SOURCE.getMessage(code, strings, locale());
        } catch (NoSuchMessageException e) {
            return code;
        }
    }

    /** Выбор из трёх вариантов значения справочника; непереведённое значение заменяется русским. */
    public static String pick(String ru, String en, String be) {
        String language = locale().getLanguage();
        String value = "en".equals(language) ? en : "be".equals(language) ? be : ru;
        return value == null || value.isBlank() ? ru : value;
    }

    private Messages() {
    }
}

```

Листинг А.12 –

bank-common/src/test/java/by/bsuir/bank/common/i18n/MessagesTest.java

```

package by.bsuir.bank.common.i18n;

import by.bsuir.bank.common.HttpApis;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.springframework.context.i18n.LocaleContextHolder;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpMethod;
import org.springframework.http.MediaType;
import org.springframework.test.web.client.MockRestServiceServer;
import org.springframework.web.client.RestClient;

import java.util.Locale;

import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.client.match.MockRestRequestMatchers.header;
import static org.springframework.test.web.client.match.MockRestRequestMatchers.method;
import static org.springframework.test.web.client.match.MockRestRequestMatchers.requestTo;
import static org.springframework.test.web.client.response.MockRestResponseCreators.withSuccess;

/** Модульные тесты локализации: выбор языка, подстановка аргументов, коды хранимых текстов, передача языка. */
class MessagesTest {

    @AfterEach
    void resetLocale() {
        LocaleContextHolder.resetLocaleContext();
    }

    @ParameterizedTest(name = "{0}: {1}")
    @CsvSource({
        "ru, Обязательное поле, 'Касса банка, BYN'",
        "en, Required field, 'Bank cash desk, BYN'",
        "be, Абавязковае поле, 'Каса банка, BYN'",
        "en-GB, Required field, 'Bank cash desk, BYN'",
        "de, Обязательное поле, 'Касса банка, BYN'",
    })
    @DisplayName("Текст выбирается по языку запроса, неподдерживаемый язык заменяется русским")
    void messageByLocale(String language, String required, String cash) {
        LocaleContextHolder.setLocale(Locale.forLanguageTag(language));
    }
}

```

```

        assertThat(Messages.get("validation.required")).isEqualTo(required);
        assertThat(Messages.get("account.cash", "BYN")).isEqualTo(cash);
    }

    @Test
    @DisplayName("Вне запроса используется русский язык, неизвестный ключ возвращается как есть")
    void defaults() {
        assertThat(Messages.locale()).isEqualTo(Messages.RU);
        assertThat(Messages.get("validation.required")).isEqualTo("Обязательное поле");
        assertThat(Messages.get("no.such.key")).isEqualTo("no.such.key");
    }

    @Test
    @DisplayName("Числа подставляются без переформатирования")
    void argumentsAreNotReformatted() {
        assertThat(Messages.get("posting.atm.payment", "МТС", "0297654321"))
            .isEqualTo("Оплата услуг связи МТС, номер 0297654321");
        assertThat(Messages.get("account.cash", 1234567.5)).isEqualTo("Касса банка, 1234567.5");
    }

    @Test
    @DisplayName("Значение справочника выбирается из трёх столбцов; пустой перевод заменяется русским")
    void pick() {
        LocaleContextHolder.setLocale(Messages.BE);
        assertThat(Messages.pick("Минск", "Minsk", "Мінск")).isEqualTo("Мінск");
        assertThat(Messages.pick("Минск", "Minsk", null)).isEqualTo("Минск");

        LocaleContextHolder.setLocale(Messages.EN);
        assertThat(Messages.pick("Минск", "Minsk", "Minsk")).isEqualTo("Minsk");
    }

    @Test
    @DisplayName("Хранимый код раскрывается на языке читающего, обычный текст не меняется")
    void localizedCodes() {
        String posting = Localized.code("posting.deposit.accrual");
        String payment = Localized.code("posting.atm.payment", "A1", "0291112233");
        assertThat(posting).isEqualTo("@posting.deposit.accrual");
        assertThat(payment).isEqualTo("@posting.atm.payment|A1|0291112233");

        assertThat(Localized.render(posting)).isEqualTo("Начисление процентов по депозиту");
        assertThat(Localized.render("Иванов Иван Иванович")).isEqualTo("Иванов Иван Иванович");
        assertThat(Localized.render(null)).isNull();

        LocaleContextHolder.setLocale(Messages.EN);
        assertThat(Localized.render(posting)).isEqualTo("Deposit interest accrued");
        assertThat(Localized.render(payment)).isEqualTo("Mobile service payment A1, number 0291112233");

        LocaleContextHolder.setLocale(Messages.BE);
        assertThat(Localized.render(posting)).isEqualTo("Налічэнне праэнтаў па дэпазіце");
    }

    @Test
    @DisplayName("Язык пользователя передаётся смежному сервису в заголовке Accept-Language")
    void languageIsPropagated() {
        RestClient.Builder builder = HttpApis.withLanguage(RestClient.builder());
        MockRestServiceServer server = MockRestServiceServer.bindTo(builder).build();
        server.expect(requestTo("http://ledger/api/bank-day"))
            .andExpect(method(HttpMethod.GET))
            .andExpect(header(HttpHeaders.ACCEPT_LANGUAGE, "be"))
            .andRespond(withSuccess("{} ", MediaType.APPLICATION_JSON));
        LocaleContextHolder.setLocale(Messages.BE);

        builder.build().get().uri("http://ledger/api/bank-day").retrieve().toBodilessEntity();

        server.verify();
    }
}

```

Микросервис «Клиенты» (client-service)

Листинг А.13 – client-service/pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">

```

```

<modelVersion>4.0.0</modelVersion>

<parent>
  <groupId>by.bsuir.bank</groupId>
  <artifactId>bank</artifactId>
  <version>1.0.0</version>
</parent>

<artifactId>client-service</artifactId>
<name>client-service</name>
<description>МРП1. Микросервис «Клиенты»: ввод и модификация данных о клиентах банка</description>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <!-- из общей библиотеки используется только локализация: пакет by.bsuir.bank.common.i18n -->
  <dependency>
    <groupId>by.bsuir.bank</groupId>
    <artifactId>bank-common</artifactId>
  </dependency>
  <dependency>
    <groupId>by.bsuir.bank</groupId>
    <artifactId>ui-kit</artifactId>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-failsafe-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

Листинг А.14 – client-service/src/main/resources/application.yml

```

server:
  port: ${CLIENT_SERVICE_PORT:8081}

spring:
  application:
    name: client-service
  datasource:

```

```
# База в памяти: создаётся и заполняется при каждом старте (schema.sql + data.sql)
url: jdbc:h2:mem:clients;DB_CLOSE_DELAY=-1
username: sa

sql:
  init:
    mode: always
jpa:
  hibernate:
    ddl-auto: validate
    open-in-view: false
h2:
  console:
    enabled: true          # http://localhost:8081/h2-console – проверка таблиц «прямо в базе»
jackson:
  deserialization:
    fail-on-unknown-properties: false
```

Листинг А.15 – client-service/src/main/resources/schema.sql

```
-- Схема данных модуля «Клиенты». Поля типа «Список» вынесены в отдельные справочники;
-- наименования справочников хранятся на трёх языках: русском (name), английском и белорусском.

create table city (
  id          bigint generated by default as identity primary key,
  name        varchar(100) not null unique,
  name_en     varchar(100) not null,
  name_be     varchar(100) not null
);

create table marital_status (
  id          bigint generated by default as identity primary key,
  name        varchar(100) not null unique,
  name_en     varchar(100) not null,
  name_be     varchar(100) not null
);

create table citizenship (
  id          bigint generated by default as identity primary key,
  name        varchar(100) not null unique,
  name_en     varchar(100) not null,
  name_be     varchar(100) not null
);

create table disability (
  id          bigint generated by default as identity primary key,
  name        varchar(100) not null unique,
  name_en     varchar(100) not null,
  name_be     varchar(100) not null
);

create table client (
  id          bigint generated by default as identity primary key,
  last_name   varchar(60) not null,
  first_name  varchar(60) not null,
  middle_name varchar(60) not null,
  birth_date  date        not null,
  sex         varchar(1)  not null check (sex in ('M', 'F')),
  passport_series varchar(2) not null,
  passport_number varchar(7) not null,
  issued_by   varchar(200) not null,
  issue_date  date        not null,
  identification_number varchar(14) not null,
  birth_place varchar(200) not null,
  residence_city_id bigint    not null references city,
  residence_address varchar(200) not null,
  home_phone   varchar(20),
  mobile_phone varchar(20),
  email        varchar(100),
  registration_city_id bigint    not null references city,
  marital_status_id bigint    not null references marital_status,
  citizenship_id bigint    not null references citizenship,
  disability_id bigint    not null references disability,
  pensioner    boolean      not null,
  monthly_income decimal(14, 2),
  -- последний рубеж защиты от дублей: даже в обход сервиса два одинаковых паспорта не записать
  constraint uq_client_passport unique (passport_series, passport_number),
  constraint uq_client_identification unique (identification_number)
);

create index ix_client_fio on client (last_name, first_name, middle_name);
```

Листинг А.16 – client-service/src/main/resources/data.sql

```
-- Справочники: наименование на русском, английском и белорусском языках
insert into city (name, name_en, name_be) values
    ('Минск', 'Minsk', 'Мінск'),
    ('Брест', 'Brest', 'Брэст'),
    ('Витебск', 'Vitebsk', 'Віцебск'),
    ('Гомель', 'Gomel', 'Гомель'),
    ('Гродно', 'Grodno', 'Гродна'),
    ('Могилёв', 'Mogilev', 'Магілёў'),
    ('Барановичи', 'Baranovichi', 'Баранавічы'),
    ('Борисов', 'Borisov', 'Барысаў');

insert into marital_status (name, name_en, name_be) values
    ('Холост / не замужем', 'Single', 'Халасты / незамужняя'),
    ('Женат / замужем', 'Married', 'Жанаты / замужам'),
    ('Разведён / разведена', 'Divorced', 'Разведзены / разведзеная'),
    ('Вдовец / вдова', 'Widowed', 'Удавец / удава');

insert into citizenship (name, name_en, name_be) values
    ('Республика Беларусь', 'Republic of Belarus', 'Рэспубліка Беларусь'),
    ('Российская Федерация', 'Russian Federation', 'Расійская Федэрацыя'),
    ('Республика Казахстан', 'Republic of Kazakhstan', 'Рэспубліка Казахстан'),
    ('Украина', 'Ukraine', 'Украіна'),
    ('Литовская Республика', 'Republic of Lithuania', 'Літоўская Рэспубліка');

insert into disability (name, name_en, name_be) values
    ('Нет', 'None', 'Няма'),
    ('I группа', 'Group I', 'I група'),
    ('II группа', 'Group II', 'II група'),
    ('III группа', 'Group III', 'III група');

-- Тестовые клиенты (паспортные данные вымышленные)
insert into client (last_name, first_name, middle_name, birth_date, sex, passport_series, passport_number,
    issued_by, issue_date, identification_number, birth_place, residence_city_id,
    residence_address, home_phone, mobile_phone, email, registration_city_id,
    marital_status_id, citizenship_id, disability_id, pensioner, monthly_income)
values ('Лебедев', 'Артём', 'Владимирович', date '2001-03-14', 'М', 'МР', '3141592',
    'Советское РУВД г. Минска', date '2019-04-02', '3140301A001PB5', 'г. Минск', 1,
    'ул. Петруся Бровки, д. 6, кв. 12', '293-88-44', '+375 (29) 314-15-92', 'lebedevich@example.by', 1,
    1, 1, 1, false, 4200.00),
    ('Савицкая', 'Марина', 'Олеговна', date '1994-11-05', 'Ф', 'МС', '2718281',
    'Московское РУВД г. Минска', date '2020-01-17', '4051194B014PB1', 'г. Борисов', 1,
    'пр. Дзержинского, д. 91, кв. 205', null, '+375 (33) 271-82-81', 'savitskaya@example.by', 8,
    2, 1, 1, false, 3100.50),
    ('Ковальчук', 'Игорь', 'Петрович', date '1958-07-23', 'М', 'АВ', '1618033',
    'Ленинский РОВД г. Бреста', date '2014-08-11', '3230758C027PB8', 'д. Чернавичи Брестского р-на', 2,
    'ул. Московская, д. 273, кв. 41', '205-17-33', null, null, 2,
    2, 1, 4, true, 1180.00),
    ('Абрамович', 'Елена', 'Сергеевна', date '1987-02-28', 'Ф', 'КН', '1414213',
    'Октябрьский РОВД г. Гродно', date '2017-06-30', '4280287K003PB2', 'г. Гродно', 5,
    'ул. Ожешко, д. 15, кв. 7', null, '+375 (44) 141-42-13', 'abramovich@example.by', 5,
    3, 1, 1, false, null),
    ('Жуков', 'Дмитрий', 'Александрович', date '1979-09-09', 'М', 'ВМ', '1732050',
    'Железнодорожный РОВД г. Гомеля', date '2016-03-21', '3090979E045PB6', 'г. Жлобин', 4,
    'ул. Советская, д. 97, кв. 118', '255-40-12', '+375 (25) 173-20-50', null, 1,
    2, 2, 1, false, 5600.00),
    ('Шевчук', 'Ольга', 'Николаевна', date '1951-12-01', 'Ф', 'ВМ', '2236067',
    'Первомайский РОВД г. Витебска', date '2012-10-04', '4011251H019PB3', 'г. Полоцк', 3,
    'пр. Фрунзе, д. 34, кв. 56', '236-70-98', null, null, 3,
    4, 1, 3, true, 920.40);
```

Листинг А.17 – client-service/src/main/resources/messages.properties

```
# Тексты сервиса «Клиенты» – русский язык (по умолчанию)
validation.lettersOnly=Допустимы только буквы
validation.tooLong=Слишком длинное значение
validation.date=Укажите существующую дату ДД.ММ.ГГГГ не позднее сегодняшней
validation.sex=Выберите пол
validation.passportSeries=Серия – две заглавные латинские буквы
validation.passportNumber=Номер паспорта – семь цифр
validation.identificationNumber=Формат: 7 цифр, буква, 3 цифры, 2 буквы, цифра
validation.homePhone=Формат: 293-88-44
validation.mobilePhone=Формат: +375 (29) 314-15-92
validation.email=Некорректный e-mail
validation.incomeNegative=Доход не может быть отрицательным
validation.incomeFormat=Денежная сумма: до 12 цифр и не более 2 знаков после запятой

client.issueBeforeBirth=Паспорт не может быть выдан раньше даты рождения
client.duplicatePassport=Клиент с таким паспортом уже зарегистрирован
client.duplicateIdentification=Клиент с таким идентификационным номером уже зарегистрирован
```

```

client.duplicate=Такой клиент уже есть в базе
client.unknownDictionaryValue=Значение отсутствует в справочнике
client.notFound=Клиент с идентификатором {0} не найден
client.invalidValue=Некорректное значение
client.integrity=Клиент с таким паспортом или идентификационным номером уже есть в базе

```

Листинг A.18 – client-service/src/main/resources/messages_be.properties

```

# Тэксты сэрвісу «Кліенты» – беларуская мова
validation.lettersOnly=Допушчальныя толькі літары
validation.tooLong=Занадта доўгае значэнне
validation.date=Укажыце існуючую дату ДД.ММ.ГГГГ не пазней за сённяшняю
validation.sex=Выберыце пол
validation.passportSeries=Серыя – дзве вялікія лацінскія літары
validation.passportNumber=Нумар пашпарта – сем лічбаў
validation.identificationNumber=Фармат: 7 лічбаў, літара, 3 лічбы, 2 літары, лічба
validation.homePhone=Фармат: 293-88-44
validation.mobilePhone=Фармат: +375 (29) 314-15-92
validation.email=Некарэктны e-mail
validation.incomeNegative=Даход не можа быць адмоўным
validation.incomeFormat=Грашовая сума: да 12 лічбаў і не больш за 2 знакі пасля коскі

client.issueBeforeBirth=Пашпарт не можа быць выдадзены раней за дату нараджэння
client.duplicatePassport=Кліент з такім пашпартам ужо зарэгістраваны
client.duplicateIdentification=Кліент з такім ідэнтыфікацыйным нумарам ужо зарэгістраваны
client.duplicate=Такі кліент ужо ёсць у базе
client.unknownDictionaryValue=Значэнне адсутнічае ў даведніку
client.notFound=Кліент з ідэнтыфікатарам {0} не знойдзены
client.invalidValue=Некарэктнае значэнне
client.integrity=Кліент з такім пашпартам або ідэнтыфікацыйным нумарам ужо ёсць у базе

```

Листинг A.19 – client-service/src/main/resources/messages_en.properties

```

# Client service texts – English
validation.lettersOnly=Letters only
validation.tooLong=The value is too long
validation.date=Enter an existing date DD.MM.YYYY not later than today
validation.sex=Select the sex
validation.passportSeries=Series: two capital Latin letters
validation.passportNumber=Passport number: seven digits
validation.identificationNumber=Format: 7 digits, letter, 3 digits, 2 letters, digit
validation.homePhone=Format: 293-88-44
validation.mobilePhone=Format: +375 (29) 314-15-92
validation.email=Invalid e-mail
validation.incomeNegative=Income cannot be negative
validation.incomeFormat=Amount: up to 12 digits and no more than 2 decimal places

client.issueBeforeBirth=The passport cannot be issued before the date of birth
client.duplicatePassport=A client with this passport is already registered
client.duplicateIdentification=A client with this identification number is already registered
client.duplicate=This client is already in the database
client.unknownDictionaryValue=The value is missing from the dictionary
client.notFound=Client with id {0} not found
client.invalidValue=Invalid value
client.integrity=A client with this passport or identification number already exists

```

Листинг A.20 – client-service/src/main/java/by/bsuir/bank/client/domain/Citizenship.java

```

package by.bsuir.bank.client.domain;

import jakarta.persistence.Entity;

/** Справочник: Гражданство. */
@Entity
public class Citizenship extends DictionaryItem {
}

```

Листинг A.21 – client-service/src/main/java/by/bsuir/bank/client/domain/City.java

```

package by.bsuir.bank.client.domain;

import jakarta.persistence.Entity;

```

```

/** Справочник: Города (проживания и прописки). */
@Entity
public class City extends DictionaryItem {
}

```

Листинг А.22 – client-service/src/main/java/by/bsuir/bank/client/domain/Client.java

```

package by.bsuir.bank.client.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDate;

/** Клиент банка – физическое лицо. */
@Entity
@Getter
@Setter
public class Client {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String lastName;
    private String firstName;
    private String middleName;
    private LocalDate birthDate;

    @Enumerated(EnumType.STRING)
    private Sex sex;

    private String passportSeries;
    private String passportNumber;
    private String issuedBy;
    private LocalDate issueDate;
    private String identificationNumber;
    private String birthPlace;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private City residenceCity;

    private String residenceAddress;
    private String homePhone;
    private String mobilePhone;
    private String email;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private City registrationCity;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private MaritalStatus maritalStatus;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private Citizenship citizenship;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private Disability disability;

    private boolean pensioner;
    private BigDecimal monthlyIncome;

    public String getFullName() {
        return lastName + " " + firstName + " " + middleName;
    }
}

```

Листинг А.23 –

client-service/src/main/java/by/bsuir/bank/client/domain/DictionaryItem.java

```
package by.bsuir.bank.client.domain;

import by.bsuir.bank.common.i18n.Messages;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.MappedSuperclass;
import lombok.AccessLevel;
import lombok.Getter;
import lombok.Setter;

/** Общая структура справочников: идентификатор и наименование на трёх языках. */
@MappedSuperclass
@Getter
@Setter
public abstract class DictionaryItem {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Getter(AccessLevel.NONE)
    private String nameEn;

    @Getter(AccessLevel.NONE)
    private String nameBe;

    /** Наименование на языке запроса: именно оно попадает в ответы REST. */
    public String getName() {
        return Messages.pick(name, nameEn, nameBe);
    }
}
```

Листинг А.24 –

client-service/src/main/java/by/bsuir/bank/client/domain/Disability.java

```
package by.bsuir.bank.client.domain;

import jakarta.persistence.Entity;

/** Справочник: Группы инвалидности. */
@Entity
public class Disability extends DictionaryItem {
}
```

Листинг А.25 –

client-service/src/main/java/by/bsuir/bank/client/domain/MaritalStatus.java

```
package by.bsuir.bank.client.domain;

import jakarta.persistence.Entity;

/** Справочник: Семейное положение. */
@Entity
public class MaritalStatus extends DictionaryItem {
}
```

Листинг А.26 –

client-service/src/main/java/by/bsuir/bank/client/domain/Sex.java

```
package by.bsuir.bank.client.domain;

public enum Sex {
    M, F
}
```

Листинг А.27 – client-

service/src/main/java/by/bsuir/bank/client/repository/CitizenshipRepository.java

```
package by.bsuir.bank.client.repository;

import by.bsuir.bank.client.domain.Citizenship;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CitizenshipRepository extends JpaRepository<Citizenship, Long> {
}
```

Листинг А.28 – client-service/src/main/java/by/bsuir/bank/client/repository/CityRepository.java

```
package by.bsuir.bank.client.repository;

import by.bsuir.bank.client.domain.City;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CityRepository extends JpaRepository<City, Long> {
}
```

Листинг А.29 – client- service/src/main/java/by/bsuir/bank/client/repository/ClientRepository.java

```
package by.bsuir.bank.client.repository;

import by.bsuir.bank.client.domain.Client;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;

public interface ClientRepository extends JpaRepository<Client, Long> {

    /** Список клиентов, отсортированный по фамилии (затем по имени и отчеству). */
    @EntityGraph(attributePaths = {"residenceCity", "registrationCity", "maritalStatus", "citizenship", "disability"})
    List<Client> findAllByOrderByLastNameAscFirstNameAscMiddleNameAsc();

    @EntityGraph(attributePaths = {"residenceCity", "registrationCity", "maritalStatus", "citizenship", "disability"})
    Optional<Client> findWithDictionariesById(Long id);

    Optional<Client> findByPassportSeriesAndPassportNumber(String series, String number);

    Optional<Client> findByIdentificationNumber(String identificationNumber);
}
```

Листинг А.30 – client- service/src/main/java/by/bsuir/bank/client/repository/DisabilityRepository.java

```
package by.bsuir.bank.client.repository;

import by.bsuir.bank.client.domain.Disability;
import org.springframework.data.jpa.repository.JpaRepository;

public interface DisabilityRepository extends JpaRepository<Disability, Long> {
}
```

Листинг А.31 – client- service/src/main/java/by/bsuir/bank/client/repository/MaritalStatusRepository.java

```
package by.bsuir.bank.client.repository;

import by.bsuir.bank.client.domain.MaritalStatus;
import org.springframework.data.jpa.repository.JpaRepository;

public interface MaritalStatusRepository extends JpaRepository<MaritalStatus, Long> {
}
```

Листинг А.32 – client-service/src/main/java/by/bsuir/bank/client/dto/ApiError.java

```

package by.bsuir.bank.client.dto;

import java.util.Map;

/** Единый формат ошибки: общее сообщение и сообщения по полям формы. */
public record ApiError(String message, Map<String, String> fields) {
}

```

Листинг А.33 – client-service/src/main/java/by/bsuir/bank/client/dto/ClientRequest.java

```

package by.bsuir.bank.client.dto;

import by.bsuir.bank.client.validation.DateText;
import by.bsuir.bank.client.validation.Masks;
import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Digits;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

import java.math.BigDecimal;

/**
 * Данные формы клиента. Серверная валидация (вторая линия после проверки в браузере):
 * обязательность – @NotBlank/@NotNull, маски – @Pattern, даты – @DateText.
 * Сообщения заданы ключами {...}: текст берётся из messages*.properties на языке запроса.
 */
public record ClientRequest(

    @NotBlank(message = REQUIRED) @Size(max = 60, message = TOO_LONG)
    @Pattern(regexp = Masks.NAME, message = LETTERS_ONLY)
    String lastName,

    @NotBlank(message = REQUIRED) @Size(max = 60, message = TOO_LONG)
    @Pattern(regexp = Masks.NAME, message = LETTERS_ONLY)
    String firstName,

    @NotBlank(message = REQUIRED) @Size(max = 60, message = TOO_LONG)
    @Pattern(regexp = Masks.NAME, message = LETTERS_ONLY)
    String middleName,

    @NotBlank(message = REQUIRED) @DateText
    String birthDate,

    @NotBlank(message = REQUIRED) @Pattern(regexp = "^[MF]$", message = "{validation.sex}")
    String sex,

    @NotBlank(message = REQUIRED)
    @Pattern(regexp = Masks.PASSPORT_SERIES, message = "{validation.passportSeries}")
    String passportSeries,

    @NotBlank(message = REQUIRED)
    @Pattern(regexp = Masks.PASSPORT_NUMBER, message = "{validation.passportNumber}")
    String passportNumber,

    @NotBlank(message = REQUIRED) @Size(max = 200, message = TOO_LONG)
    String issuedBy,

    @NotBlank(message = REQUIRED) @DateText
    String issueDate,

    @NotBlank(message = REQUIRED)
    @Pattern(regexp = Masks.IDENTIFICATION_NUMBER, message = "{validation.identificationNumber}")
    String identificationNumber,

    @NotBlank(message = REQUIRED) @Size(max = 200, message = TOO_LONG)
    String birthPlace,

    @NotNull(message = REQUIRED)
    Long residenceCityId,

    @NotBlank(message = REQUIRED) @Size(max = 200, message = TOO_LONG)
    String residenceAddress,

    @Pattern(regexp = Masks.HOME_PHONE, message = "{validation.homePhone}")
    String homePhone,

```

```

@Pattern(regexp = Masks.MOBILE_PHONE, message = "{validation.mobilePhone}")
String mobilePhone,

@Size(max = 100, message = TOO_LONG) @Pattern(regexp = Masks.EMAIL, message = "{validation.email}")
String email,

@NotNull(message = REQUIRED)
Long registrationCityId,

@NotNull(message = REQUIRED)
Long maritalStatusId,

@NotNull(message = REQUIRED)
Long citizenshipId,

@NotNull(message = REQUIRED)
Long disabilityId,

@NotNull(message = REQUIRED)
Boolean pensioner,

@DecimalMin(value = "0.00", message = "{validation.incomeNegative}")
@Digits(integer = 12, fraction = 2, message = "{validation.incomeFormat}")
BigDecimal monthlyIncome
) {

public static final String REQUIRED = "{validation.required}";
public static final String TOO_LONG = "{validation.tooLong}";
public static final String LETTERS_ONLY = "{validation.lettersOnly}";

/** Пробелы по краям отбрасываются до проверок: «пробел вместо имени» превращается в пустую строку. */
public ClientRequest {
    lastName = trim(lastName);
    firstName = trim(firstName);
    middleName = trim(middleName);
    birthDate = trim(birthDate);
    passportSeries = trim(passportSeries);
    passportNumber = trim(passportNumber);
    issuedBy = trim(issuedBy);
    issueDate = trim(issueDate);
    identificationNumber = trim(identificationNumber);
    birthPlace = trim(birthPlace);
    residenceAddress = trim(residenceAddress);
    homePhone = trim(homePhone);
    mobilePhone = trim(mobilePhone);
    email = trim(email);
}

private static String trim(String value) {
    return value == null ? null : value.trim();
}
}

```

Листинг А.34 – client-service/src/main/java/by/bsuir/bank/client/dto/ClientResponse.java

```

package by.bsuir.bank.client.dto;

import by.bsuir.bank.client.domain.Client;
import by.bsuir.bank.client.validation.Dates;

import java.math.BigDecimal;

public record ClientResponse(
    Long id,
    String fullName,
    String lastName,
    String firstName,
    String middleName,
    String birthDate,
    String sex,
    String passportSeries,
    String passportNumber,
    String issuedBy,
    String issueDate,
    String identificationNumber,
    String birthPlace,
    Long residenceCityId,
    String residenceCity,
    String residenceAddress,

```

```

        String homePhone,
        String mobilePhone,
        String email,
        Long registrationCityId,
        String registrationCity,
        Long maritalStatusId,
        String maritalStatus,
        Long citizenshipId,
        String citizenship,
        Long disabilityId,
        String disability,
        boolean pensioner,
        BigDecimal monthlyIncome
    ) {

        public static ClientResponse from(Client c) {
            return new ClientResponse(
                c.getId(), c.getFullName(), c.getLastName(), c.getFirstName(), c.getMiddleName(),
                Dates.format(c.getBirthDate()), c.getSex().name(),
                c.getPassportSeries(), c.getPassportNumber(), c.getIssuedBy(), Dates.format(c.getIssueDate()),
                c.getIdentificationNumber(), c.getBirthPlace(),
                c.getResidenceCity().getId(), c.getResidenceCity().getName(), c.getResidenceAddress(),
                c.getHomePhone(), c.getMobilePhone(), c.getEmail(),
                c.getRegistrationCity().getId(), c.getRegistrationCity().getName(),
                c.getMaritalStatus().getId(), c.getMaritalStatus().getName(),
                c.getCitizenship().getId(), c.getCitizenship().getName(),
                c.getDisability().getId(), c.getDisability().getName(),
                c.isPensioner(), c.getMonthlyIncome());
        }
    }
}

```

Листинг А.35 – client-service/src/main/java/by/bsuir/bank/client/dto/Dictionaries.java

```

package by.bsuir.bank.client.dto;

import by.bsuir.bank.client.domain.Citizenship;
import by.bsuir.bank.client.domain.City;
import by.bsuir.bank.client.domain.Disability;
import by.bsuir.bank.client.domain.MaritalStatus;

import java.util.List;

/** Содержимое всех справочников – для выпадающих списков формы. */
public record Dictionaries(
    List<City> cities,
    List<MaritalStatus> maritalStatuses,
    List<Citizenship> citizenships,
    List<Disability> disabilities
) {
}

```

Листинг А.36 – client-service/src/main/java/by/bsuir/bank/client/validation/DateText.java

```

package by.bsuir.bank.client.validation;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

/** Строка содержит существующую календарную дату ДД.ММ.ГГГГ в диапазоне от 01.01.1900 до сегодняшнего дня. */
@Documented
@Constraint(validatedBy = DateTextValidator.class)
@Target({ElementType.FIELD, ElementType.RECORD_COMPONENT})
@Retention(RetentionPolicy.RUNTIME)
public @interface DateText {

    String message() default "{validation.date}";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

```
}
```

Листинг А.37 – client-service/src/main/java/by/bsuir/bank/client/validation/DateTextValidator.java

```
package by.bsuir.bank.client.validation;

import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;

import java.time.LocalDate;

public class DateTextValidator implements ConstraintValidator<DateText, String> {

    private static final LocalDate MIN = LocalDate.of(1900, 1, 1);

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        if (value == null || value.isBlank()) {
            return true; // обязательность проверяет @NotBlank
        }
        return Dates.parse(value)
            .filter(date -> !date.isBefore(MIN) && !date.isAfter(LocalDate.now()))
            .isPresent();
    }
}
```

Листинг А.38 – client-service/src/main/java/by/bsuir/bank/client/validation/Dates.java

```
package by.bsuir.bank.client.validation;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.format.ResolverStyle;
import java.util.Optional;

/** Даты передаются между клиентом и сервисом текстом в формате ДД.ММ.ГГГГ. */
public final class Dates {

    /** STRICT отвергает 31.02.2015 и 29.02.2015 – без него парсер молча «исправил» бы дату на 28.02. */
    public static final DateTimeFormatter FORMAT =
        DateTimeFormatter.ofPattern("dd.MM.uuuu").withResolverStyle(ResolverStyle.STRICT);

    public static Optional<LocalDate> parse(String text) {
        if (text == null) {
            return Optional.empty();
        }
        try {
            return Optional.of(LocalDate.parse(text.trim(), FORMAT));
        } catch (DateTimeParseException e) {
            return Optional.empty();
        }
    }

    public static String format(LocalDate date) {
        return date == null ? null : FORMAT.format(date);
    }

    private Dates() {
    }
}
```

Листинг А.39 – client-service/src/main/java/by/bsuir/bank/client/validation/Masks.java

```
package by.bsuir.bank.client.validation;

/** Маски полей клиента. Те же выражения продублированы в web-клиенте (static/app.js). */
public final class Masks {

    /** ФИО: только буквы, составные части через дефис, пробел или апостроф. */
    public static final String NAME = "[A-Яа-яЁёА-Зa-z]{1,30}";
}
```

```

/** Серия паспорта: две заглавные латинские буквы (MP, MC, AB, BM, KH, HB, KB, PP...). */
public static final String PASSPORT_SERIES = "^[A-Z]{2}$";

/** Номер паспорта: семь цифр. */
public static final String PASSPORT_NUMBER = "^\\d{7}$";

/** Идентификационный номер: 7 цифр, буква, 3 цифры, 2 буквы, цифра – 3140301A001PB5. */
public static final String IDENTIFICATION_NUMBER = "^\\d{7}[A-Z]\\d{3}[A-Z]{2}\\d$";

/** Домашний телефон: 293-88-44. Пустая строка допустима – поле необязательное. */
public static final String HOME_PHONE = "^$|^\\d{3}-\\d{2}-\\d{2}$";

/** Мобильный телефон: +375 (29) 314-15-92, коды операторов 25, 29, 33, 44. */
public static final String MOBILE_PHONE = "^$|^\\+375 \\((25|29|33|44)\\) \\d{3}-\\d{2}-\\d{2}$";

public static final String EMAIL = "^$|^([A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.([A-Za-z]{2,})$)";

private Masks() {
}
}

```

Листинг А.40 – client-service/src/main/java/by/bsuir/bank/client/ClientServiceApplication.java

```

package by.bsuir.bank.client;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

// вместе с пакетом сервиса сканируется локализация из общей библиотеки
@SpringBootApplication(scanBasePackages = {"by.bsuir.bank.client", "by.bsuir.bank.common.i18n"})
public class ClientServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(ClientServiceApplication.class, args);
    }
}

```

Листинг А.41 – client-service/src/main/java/by/bsuir/bank/client/service/ClientService.java

```

package by.bsuir.bank.client.service;

import by.bsuir.bank.client.domain.Client;
import by.bsuir.bank.client.domain.Sex;
import by.bsuir.bank.client.dto.ClientRequest;
import by.bsuir.bank.client.dto.ClientResponse;
import by.bsuir.bank.client.repository.CitizenshipRepository;
import by.bsuir.bank.client.repository.CityRepository;
import by.bsuir.bank.client.repository.ClientRepository;
import by.bsuir.bank.client.repository.DisabilityRepository;
import by.bsuir.bank.client.repository.MaritalStatusRepository;
import by.bsuir.bank.client.validation.Dates;
import by.bsuir.bank.common.i18n.Messages;
import lombok.RequiredArgsConstructor;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

@Service
@RequiredArgsConstructor
public class ClientService {

    private final ClientRepository clients;
    private final CityRepository cities;
    private final MaritalStatusRepository maritalStatuses;
    private final CitizenshipRepository citizenships;
    private final DisabilityRepository disabilities;

    @Transactional(readOnly = true)
    public List<ClientResponse> findAll() {
        return clients.findAllByOrderByLastNameAscFirstNameAscMiddleNameAsc().stream()
    }
}

```

```

        .map(ClientResponse::from)
        .toList();
    }

    @Transactional(readOnly = true)
    public ClientResponse get(Long id) {
        return clients.findWithDictionariesById(id)
            .map(ClientResponse::from)
            .orElseThrow(() -> notFound(id));
    }

    @Transactional
    public ClientResponse create(ClientRequest request) {
        return save(new Client(), request);
    }

    @Transactional
    public ClientResponse update(Long id, ClientRequest request) {
        return save(clients.findById(id).orElseThrow(() -> notFound(id)), request);
    }

    @Transactional
    public void delete(Long id) {
        if (!clients.existsById(id)) {
            throw notFound(id);
        }
        clients.deleteById(id);
        clients.flush();
    }

    private ClientResponse save(Client client, ClientRequest request) {
        LocalDate birthDate = Dates.parse(request.birthDate()).orElseThrow();
        LocalDate issueDate = Dates.parse(request.issueDate()).orElseThrow();
        if (issueDate.isBefore(birthDate)) {
            throw new FieldErrorException(HttpStatus.BAD_REQUEST, Messages.get("error.checkFields"),
                Map.of("issueDate", Messages.get("client.issueBeforeBirth")));
        }
        checkUnique(request, client.getId());

        client.setLastName(request.lastName());
        client.setFirstName(request.firstName());
        client.setMiddleName(request.middleName());
        client.setBirthDate(birthDate);
        client.setSex(Sex.valueOf(request.sex()));
        client.setPassportSeries(request.passportSeries());
        client.setPassportNumber(request.passportNumber());
        client.setIssuedBy(request.issuedBy());
        client.setIssueDate(issueDate);
        client.setIdentificationNumber(request.identificationNumber());
        client.setBirthPlace(request.birthPlace());
        client.setResidenceCity(reference(cities, request.residenceCityId(), "residenceCityId"));
        client.setResidenceAddress(request.residenceAddress());
        client.setHomePhone(blankToNull(request.homePhone()));
        client.setMobilePhone(blankToNull(request.mobilePhone()));
        client.setEmail(blankToNull(request.email()));
        client.setRegistrationCity(reference(cities, request.registrationCityId(), "registrationCityId"));
        client.setMaritalStatus(reference(maritalStatuses, request.maritalStatusId(), "maritalStatusId"));
        client.setCitizenship(reference(citizenships, request.citizenshipId(), "citizenshipId"));
        client.setDisability(reference(disabilities, request.disabilityId(), "disabilityId"));
        client.setPensioner(request.pensioner());
        client.setMonthlyIncome(request.monthlyIncome());

        return ClientResponse.from(clients.saveAndFlush(client));
    }

    /** Паспорт и идентификационный номер уникальны; при редактировании собственная запись клиента не в счёт. */
    private void checkUnique(ClientRequest request, Long selfId) {
        Map<String, String> errors = new LinkedHashMap<>();
        clients.findByPassportSeriesAndPassportNumber(request.passportSeries(), request.passportNumber())
            .filter(other -> !other.getId().equals(selfId))
            .ifPresent(other -> {
                errors.put("passportSeries", Messages.get("client.duplicatePassport"));
                errors.put("passportNumber", Messages.get("client.duplicatePassport"));
            });
        clients.findByIdentificationNumber(request.identificationNumber())
            .filter(other -> !other.getId().equals(selfId))
            .ifPresent(other -> errors.put("identificationNumber", Messages.get("client.duplicateIdentification")));
        if (!errors.isEmpty()) {
            throw new FieldErrorException(HttpStatus.CONFLICT, Messages.get("client.duplicate"), errors);
        }
    }
}

```

```

private static <T> T reference(JpaRepository<T, Long> dictionary, Long id, String field) {
    return dictionary.findById(id).orElseThrow(() -> new FieldErrorException(HttpStatus.BAD_REQUEST,
        Messages.get("error.checkFields"), Map.of(field, Messages.get("client.unknownDictionaryValue"))));
}

private static String blankToNull(String value) {
    return value == null || value.isBlank() ? null : value;
}

private static NotFoundException notFound(Long id) {
    return new NotFoundException(Messages.get("client.notFound", id));
}
}

```

Листинг А.42 – client- service/src/main/java/by/bsuir/bank/client/service/FieldErrorException.java

```

package by.bsuir.bank.client.service;

import lombok.Getter;
import org.springframework.http.HttpStatus;

import java.util.Map;

/** Ошибка бизнес-правила, привязанная к полям формы (дубликат паспорта, отсутствие записи справочника...). */
@Getter
public class FieldErrorException extends RuntimeException {

    private final HttpStatus status;
    private final Map<String, String> fields;

    public FieldErrorException(HttpStatus status, String message, Map<String, String> fields) {
        super(message);
        this.status = status;
        this.fields = fields;
    }
}

```

Листинг А.43 – client- service/src/main/java/by/bsuir/bank/client/service/NotFoundException.java

```

package by.bsuir.bank.client.service;

public class NotFoundException extends RuntimeException {

    public NotFoundException(String message) {
        super(message);
    }
}

```

Листинг А.44 – client-service/src/main/java/by/bsuir/bank/client/web/ApiExceptionHandler.java

```

package by.bsuir.bank.client.web;

import by.bsuir.bank.client.dto.ApiError;
import by.bsuir.bank.client.service.FieldErrorException;
import by.bsuir.bank.client.service.NotFoundException;
import by.bsuir.bank.common.i18n.Messages;
import com.fasterxml.jackson.databind.JsonMappingException;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Set;

@RestControllerAdvice

```

```

public class ApiExceptionHandler {

    private static final Set<String> REQUIRED_CODES = Set.of("NotNull", "NotBlank");

    /** Нарушения Bean Validation: по одному сообщению на поле, незаполненное обязательное поле в приоритете. */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiError invalid(MethodArgumentNotValidException e) {
        Map<String, String> fields = new LinkedHashMap<>();
        for (FieldError error : e.getBindingResult().getFieldErrors()) {
            if (REQUIRED_CODES.contains(error.getCode()) || !fields.containsKey(error.getField())) {
                fields.put(error.getField(), error.getDefaultMessage());
            }
        }
        return new ApiError(Messages.get("error.checkFields"), fields);
    }

    /** JSON не разобран: например, в денежное поле или в идентификатор справочника пришёл текст. */
    @ExceptionHandler(HttpMessageNotReadableException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiError unreadable(HttpMessageNotReadableException e) {
        Map<String, String> fields = new LinkedHashMap<>();
        if (e.getCause() instanceof JsonMappingException mapping && !mapping.getPath().isEmpty()) {
            fields.put(mapping.getPath().get(0).getFieldName(), Messages.get("client.invalidValue"));
        }
        return new ApiError(Messages.get(fields.isEmpty() ? "error.badRequest" : "error.checkFields"), fields);
    }

    @ExceptionHandler(FieldErrorException.class)
    public ResponseEntity<ApiError> business(FieldErrorException e) {
        return ResponseEntity.status(e.getStatus()).body(new ApiError(e.getMessage(), e.getFields()));
    }

    @ExceptionHandler(NotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ApiError notFound(NotFoundException e) {
        return new ApiError(e.getMessage(), Map.of());
    }

    /** Сработало ограничение уникальности в самой базе (гонка двух одновременных запросов). */
    @ExceptionHandler(DataIntegrityViolationException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public ApiError integrity(DataIntegrityViolationException e) {
        return new ApiError(Messages.get("client.integrity"), Map.of());
    }
}

```

Листинг А.45 – client-service/src/main/java/by/bsuir/bank/client/web/ClientController.java

```

package by.bsuir.bank.client.web;

import by.bsuir.bank.client.dto.ClientRequest;
import by.bsuir.bank.client.dto.ClientResponse;
import by.bsuir.bank.client.service.ClientService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/clients")
@RequiredArgsConstructor
public class ClientController {

    private final ClientService service;

    @GetMapping
    public List<ClientResponse> list() {
        return service.findAll();
    }
}

```

```

    }

    @GetMapping("/{id}")
    public ClientResponse get(@PathVariable Long id) {
        return service.get(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public ClientResponse create(@Valid @RequestBody ClientRequest request) {
        return service.create(request);
    }

    @PutMapping("/{id}")
    public ClientResponse update(@PathVariable Long id, @Valid @RequestBody ClientRequest request) {
        return service.update(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}

```

Листинг А.46 – client-service/src/main/java/by/bsuir/bank/client/web/DictionaryController.java

```

package by.bsuir.bank.client.web;

import by.bsuir.bank.client.domain.City;
import by.bsuir.bank.client.dto.Dictionaries;
import by.bsuir.bank.client.repository.CitizenshipRepository;
import by.bsuir.bank.client.repository.CityRepository;
import by.bsuir.bank.client.repository.DisabilityRepository;
import by.bsuir.bank.client.repository.MaritalStatusRepository;
import by.bsuir.bank.common.i18n.Messages;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.text.Collator;
import java.util.Comparator;
import java.util.List;

@RestController
@RequiredArgsConstructor
public class DictionaryController {

    private final CityRepository cities;
    private final MaritalStatusRepository maritalStatuses;
    private final CitizenshipRepository citizenships;
    private final DisabilityRepository disabilities;

    @GetMapping("/api/dictionaries")
    public Dictionaries dictionaries() {
        Sort byId = Sort.by("id");
        // города сортируются по наименованию на языке запроса, а не по столбцу базы
        Collator collator = Collator.getInstance(Messages.locale());
        List<City> sorted = cities.findAll().stream()
            .sorted(Comparator.comparing(City::getName, collator))
            .toList();
        return new Dictionaries(sorted, maritalStatuses.findAll(byId),
            citizenships.findAll(byId), disabilities.findAll(byId));
    }
}

```

Листинг А.47 – client-service/src/main/resources/static/index.html

```

<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>BankEt</title>
    <link rel="stylesheet" href="/ui/bank.css">
</head>

```

```

<body>
  <div id="app" v-cloak></div>

  <script src="/webjars/vue/dist/vue.global.prod.js"></script>
  <script src="/ui/bank.js"></script>
  <script src="/ui/il8n.js"></script>
  <script src="/il8n.js"></script>
  <script src="/app.js"></script>
</body>
</html>

```

Листинг А.48 – client-service/src/main/resources/static/app.js

```

/* Web-клиент модуля «Работа с клиентами»: форма «Список клиентов» и форма добавления/редактирования. */

const t = Bank.t;
const NAME = /^[А-Яа-яЁёА-Za-z](?:[-' ][А-Яа-яЁёА-Za-z]+)*$/;
const letters = v => NAME.test(v) || t('check.letters');

function pastDate(v) {
  const date = Bank.parseDate(v);
  if (!date) return t('check.dateInvalid');
  if (date > new Date()) return t('check.dateFuture');
  if (date.getFullYear() < 1900) return t('check.dateOld');
  return true;
}

/*
 * Описание полей формы – единственное место, где заданы обязательность, маска ввода и проверка значения.
 * Первичная (клиентская) валидация: required – обязательность, mask – маска ввода, check – проверка формата.
 * Те же правила повторно проверяет сервис (ClientRequest + Masks).
 * Подписи и сообщения берутся из словаря il8n.js по ключам field.<имя>, section.<раздел>, check.<правило>.
 */
const SECTIONS = [
  { title: 'personal', fields: [
    { name: 'lastName', required: true, max: 60, check: letters },
    { name: 'firstName', required: true, max: 60, check: letters },
    { name: 'middleName', required: true, max: 60, check: letters },
    { name: 'birthDate', required: true, mask: '##.##.####', placeholder: t('placeholder.date'), check: pastDate },
    { name: 'sex', required: true, type: 'radio', options: [{ id: 'M', name: t('sex.M') }, { id: 'F', name: t('sex.F') }] },
    { name: 'birthPlace', required: true, max: 200 },
  ]},
  { title: 'passport', fields: [
    { name: 'passportSeries', required: true, mask: 'AA', placeholder: 'MP',
      check: v => /^[A-Z]{2}$/.test(v) || t('check.passportSeries') },
    { name: 'passportNumber', required: true, mask: '#####', placeholder: '1234567',
      check: v => /^[d]{7}$/.test(v) || t('check.passportNumber') },
    { name: 'identificationNumber', required: true, mask: '#####AA##', placeholder: '3140301A001PB5',
      check: v => /^[d]{7}[A-Z]\d{3}[A-Z]{2}\d$/.test(v) || t('check.identificationNumber') },
    { name: 'issuedBy', required: true, max: 200, wide: true },
    { name: 'issueDate', required: true, mask: '##.##.####', placeholder: t('placeholder.date'),
      check: (v, form) => {
        const valid = pastDate(v);
        if (valid !== true) return valid;
        const birth = Bank.parseDate(form.birthDate);
        return !birth || Bank.parseDate(v) >= birth || t('check.issueBeforeBirth');
      } },
  ]},
  { title: 'contacts', fields: [
    { name: 'residenceCityId', required: true, type: 'select', dictionary: 'cities' },
    { name: 'residenceAddress', required: true, max: 200, wide: true },
    { name: 'registrationCityId', required: true, type: 'select', dictionary: 'cities' },
    { name: 'homePhone', mask: '###-##-###', placeholder: '293-88-44',
      check: v => /^[d]{3}-\d{2}-\d{2}$/.test(v) || t('check.homePhone') },
    { name: 'mobilePhone', mask: '+375 (##) ###-##-###', placeholder: '+375 (29) 314-15-92',
      check: v => /^+375 \((25|29|33|44)\) \d{3}-\d{2}-\d{2}$/.test(v) || t('check.mobilePhone') },
    { name: 'email', max: 100, placeholder: 'name@example.by',
      check: v => /^[A-Za-z0-9._%+~]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/.test(v) || t('check.email') },
  ]},
  { title: 'social', fields: [
    { name: 'maritalStatusId', required: true, type: 'select', dictionary: 'maritalStatuses' },
    { name: 'citizenshipId', required: true, type: 'select', dictionary: 'citizenships' },
    { name: 'disabilityId', required: true, type: 'select', dictionary: 'disabilities' },
    { name: 'monthlyIncome', placeholder: '0.00',
      check: v => /^[d]{1,12}([.]\d{1,2})?$/test(v) || t('check.income') },
    { name: 'pensioner', type: 'checkbox' },
  ]},
];
const FIELDS = SECTIONS.flatMap(section => section.fields);

```

```

/* Проверка формы перед отправкой: возвращает { имяПоля: сообщение }. Пустой объект — ошибок нет. */
function validate(form) {
  const errors = {};
  for (const field of FIELDS) {
    const raw = form[field.name];
    const value = typeof raw === 'string' ? raw.trim() : raw;
    const empty = value === null || value === undefined || value === '';
    if (empty) {
      if (field.required) errors[field.name] = t('common.required');
      continue;
    }
    const result = field.check ? field.check(value, form) : true;
    if (result !== true) errors[field.name] = result;
  }
  return errors;
}

function emptyForm() {
  const form = {};
  for (const field of FIELDS) form[field.name] = field.type === 'checkbox' ? false : (field.type === 'select' ? null : '');
  return form;
}

/* Тело REST-запроса: строки без пробелов по краям, пустые необязательные поля — null, доход — число. */
function toPayload(form) {
  const payload = {};
  for (const field of FIELDS) {
    const raw = form[field.name];
    const value = typeof raw === 'string' ? raw.trim() : raw;
    payload[field.name] = value === '' ? null : value;
  }
  if (payload.monthlyIncome !== null) payload.monthlyIncome = Number(String(payload.monthlyIncome).replace(',', '.'));
  return payload;
}

const app = Bank.createApp({
  data() {
    return {
      view: 'list', // list | form
      clients: [],
      dictionaries: { cities: [], maritalStatuses: [], citizenships: [], disabilities: [] },
      sections: SECTIONS,
      form: emptyForm(),
      editingId: null,
      errors: {},
      formError: '',
      notice: '',
      loadError: '',
      saving: false,
    };
  },
  async mounted() {
    try {
      this.dictionaries = await Bank.api('GET', '/api/dictionaries');
      await this.reload();
    } catch (e) {
      this.loadError = e.message;
    }
  },
  methods: {
    async reload() {
      this.clients = await Bank.api('GET', '/api/clients');
    },
    openCreate() {
      this.form = emptyForm();
      this.editingId = null;
      this.showForm();
    },
    async openEdit(id) {
      const client = await Bank.api('GET', '/api/clients/' + id);
      const form = emptyForm();
      for (const field of FIELDS) form[field.name] = client[field.name] ?? form[field.name];
      if (client.monthlyIncome !== null) form.monthlyIncome = Number(client.monthlyIncome).toFixed(2);
      this.form = form;
      this.editingId = id;
      this.showForm();
    },
    showForm() {
      this.errors = {};
      this.formError = '';
      this.notice = '';
      this.view = 'form';
    }
  }
});

```

```

},
async save() {
  this.errors = validate(this.form);
  this.formError = Object.keys(this.errors).length ? t('common.formErrors') : '';
  if (this.formError) return window.scrollTo(0, 0);
  this.saving = true;
  try {
    const payload = toPayload(this.form);
    if (this.editingId === null) await Bank.api('POST', '/api/clients', payload);
    else await Bank.api('PUT', '/api/clients/' + this.editingId, payload);
    await this.reload();
    this.notice = this.editingId === null ? t('clients.added') : t('clients.saved');
    this.view = 'list';
  } catch (e) {
    this.errors = e.fields; // сообщения серверной валидации показываются у тех же полей
    this.formError = e.message;
    window.scrollTo(0, 0);
  } finally {
    this.saving = false;
  }
},
async remove(client) {
  if (!confirm(t('clients.confirmDelete', client.fullName))) return;
  try {
    await Bank.api('DELETE', '/api/clients/' + client.id);
    await this.reload();
    this.notice = t('clients.deleted');
    this.view = 'list';
  } catch (e) {
    this.notice = '';
    this.loadError = e.message;
  }
},
options(field) {
  return field.options || this.dictionaries[field.dictionary];
},
money: Bank.money,
},
template: `
<bank-nav active="clients"></bank-nav>
<main>
  <div v-if="loadError" class="alert error">{{ loadError }}</div>

  <section v-if="view === 'list'">
    <div class="toolbar">
      <div>
        <h1>{{ t('clients.list') }}</h1>
        <p class="subtitle" id="clients-count">{{ t('clients.count', clients.length) }}</p>
      </div>
      <button id="btn-add" class="primary" @click="openCreate">{{ t('clients.add') }}</button>
    </div>
    <div v-if="notice" id="notice" class="alert success">{{ notice }}</div>
    <div class="card">
      <table id="clients">
        <thead>
          <tr>
            <th>{{ t('col.fio') }}</th><th>{{ t('col.birthDate') }}</th><th>{{ t('col.passport') }}</th>
            <th>{{ t('col.identification') }}</th><th>{{ t('col.city') }}</th><th>{{ t('col.mobile') }}</th>
            <th class="num">{{ t('col.income') }}</th><th></th>
          </tr>
        </thead>
        <tbody>
          <tr v-for="client in clients" :key="client.id">
            <td class="fio">{{ client.fullName }}</td>
            <td class="nowrap">{{ client.birthDate }}</td>
            <td class="mono">{{ client.passportSeries }} {{ client.passportNumber }}</td>
            <td class="mono">{{ client.identificationNumber }}</td>
            <td>{{ client.residenceCity }}</td>
            <td class="nowrap">{{ client.mobilePhone }}</td>
            <td class="num">{{ money(client.monthlyIncome) }}</td>
            <td class="actions">
              <button class="small edit" @click="openEdit(client.id)">{{ t('clients.edit') }}</button>
              <button class="small danger delete" @click="remove(client)">{{ t('clients.delete') }}</button>
            </td>
          </tr>
        </tbody>
      </table>
      <div v-if="!clients.length" class="empty">{{ t('clients.empty') }}</div>
    </div>
  </section>

  <section v-else>

```

```

<h1>{{ editingId === null ? t('clients.new') : t('clients.editing') }}</h1>
<p class="subtitle">{{ t('clients.requiredHint') }}</p>
<div v-if="formError" id="form-alert" class="alert error">{{ formError }}</div>
<form class="card" novalidate @submit.prevent="save">
  <fieldset v-for="section in sections" :key="section.title">
    <legend>{{ t('section.' + section.title) }}</legend>
    <div class="form-grid">
      <div v-for="field in section.fields" :key="field.name"
        class="field" :class="{ invalid: errors[field.name], wide: field.wide }">
        <label :for="'f-' + field.name">{{ t('field.' + field.name) }} <span v-if="field.required" class="req">*</span></label>

        <select v-if="field.type === 'select'" :id="'f-' + field.name" v-model="form[field.name]">
          <option :value="null">{{ t('clients.choose') }}</option>
          <option v-for="item in options(field)" :key="item.id" :value="item.id">{{ item.name }}</option>
        </select>

        <div v-else-if="field.type === 'radio'" class="choices">
          <label v-for="item in options(field)" :key="item.id">
            <input type="radio" :id="'f-' + field.name + '-' + item.id" :name="field.name"
              :value="item.id" v-model="form[field.name]"> {{ item.name }}
          </label>
        </div>

        <div v-else-if="field.type === 'checkbox'" class="choices">
          <label><input type="checkbox" :id="'f-' + field.name" v-model="form[field.name]"> {{ t('clients.yes') }}</label>
        </div>

        <input v-else-if="field.mask" type="text" :id="'f-' + field.name" v-model="form[field.name]"
          v-mask="field.mask" :placeholder="field.placeholder" autocomplete="off">

        <input v-else type="text" :id="'f-' + field.name" v-model="form[field.name]"
          :maxlength="field.max" :placeholder="field.placeholder" autocomplete="off">

        <div v-if="errors[field.name]" class="error" :id="'e-' + field.name">{{ errors[field.name] }}</div>
      </div>
    </fieldset>
    <div class="form-actions">
      <button id="btn-save" type="submit" class="primary" :disabled="saving">{{ t('clients.save') }}</button>
      <button id="btn-cancel" type="button" @click="view = 'list'">{{ t('common.cancel') }}</button>
      <span style="flex: 1"></span>
      <button v-if="editingId !== null" id="btn-delete" type="button" class="danger"
        @click="remove({ id: editingId, fullName: form.lastName + ' ' + form.firstName })">{{ t('clients.deleteClient')
    ) }}</button>
    </div>
  </form>
</section>
</main>',
}, 'clients.page');

app.mount('#app');

```

Листинг А.49 – client-service/src/main/resources/static/i18n.js

```

/* Словарь web-клиента «Работа с клиентами»: русский, английский, белорусский. */
Bank.addMessages({
  ru: {
    'clients.page': 'Клиенты',
    'clients.list': 'Список клиентов',
    'clients.count': 'Всего клиентов: {0}. Сортировка по фамилии.',
    'clients.add': 'Добавить клиента',
    'clients.empty': 'Клиентов пока нет',
    'clients.edit': 'Изменить',
    'clients.delete': 'Удалить',
    'clients.deleteClient': 'Удалить клиента',
    'clients.confirmDelete': 'Удалить клиента {0}?',
    'clients.new': 'Новый клиент',
    'clients.editing': 'Редактирование клиента',
    'clients.requiredHint': 'Поля, отмеченные *, обязательны для заполнения.',
    'clients.save': 'Сохранить',
    'clients.added': 'Клиент добавлен',
    'clients.saved': 'Изменения сохранены',
    'clients.deleted': 'Клиент удалён',
    'clients.choose': '— выберите —',
    'clients.yes': 'да',
    'col.fio': 'ФИО', 'col.birthDate': 'Дата рождения', 'col.passport': 'Паспорт', 'col.identification': 'Идент. номер',
    'col.city': 'Город проживания', 'col.mobile': 'Мобильный телефон', 'col.income': 'Доход, BYN',
    'section.personal': 'Личные данные', 'section.passport': 'Паспортные данные',
    'section.contacts': 'Проживание и контакты', 'section.social': 'Социальный статус',

```

```

'field.lastName': 'Фамилия', 'field.firstName': 'Имя', 'field.middleName': 'Отчество',
'field.birthDate': 'Дата рождения', 'field.sex': 'Пол', 'field.birthPlace': 'Место рождения',
'field.passportSeries': 'Серия паспорта', 'field.passportNumber': '№ паспорта',
'field.identificationNumber': 'Идентификационный номер', 'field.issuedBy': 'Кем выдан', 'field.issueDate': 'Дата выдачи',
'field.residenceCityId': 'Город фактического проживания', 'field.residenceAddress': 'Адрес фактического проживания',
'field.registrationCityId': 'Город прописки', 'field.homePhone': 'Телефон домашний', 'field.mobilePhone': 'Телефон мобильный',
'field.email': 'E-mail', 'field.maritalStatusId': 'Семейное положение', 'field.citizenshipId': 'Гражданство',
'field.disabilityId': 'Инвалидность', 'field.monthlyIncome': 'Ежемесячный доход, BYN', 'field.pensioner': 'Пенсионер',
'sex.M': 'Мужской', 'sex.F': 'Женский',
'placeholder.date': 'ДД.ММ.ГГГГ',
'check.letters': 'Допустимы только буквы',
'check.dateInvalid': 'Такой даты не существует, формат ДД.ММ.ГГГГ',
'check.dateFuture': 'Дата не может быть в будущем',
'check.dateOld': 'Дата не может быть раньше 1900 года',
'check.passportSeries': 'Серия — две заглавные латинские буквы',
'check.passportNumber': 'Номер паспорта — семь цифр',
'check.identificationNumber': 'Формат: 7 цифр, буква, 3 цифры, 2 буквы, цифра',
'check.issueBeforeBirth': 'Паспорт не может быть выдан раньше даты рождения',
'check.homePhone': 'Формат: 293-88-44',
'check.mobilePhone': 'Формат: +375 (29) 314-15-92, код 25, 29, 33 или 44',
'check.email': 'Некорректный e-mail',
'check.income': 'Денежная сумма: только цифры и не более 2 знаков после запятой',
},
en: {
  'clients.page': 'Clients',
  'clients.list': 'Client list',
  'clients.count': 'Clients in total: {0}. Sorted by last name.',
  'clients.add': 'Add client',
  'clients.empty': 'No clients yet',
  'clients.edit': 'Edit',
  'clients.delete': 'Delete',
  'clients.deleteClient': 'Delete client',
  'clients.confirmDelete': 'Delete client {0}?',
  'clients.new': 'New client',
  'clients.editing': 'Edit client',
  'clients.requiredHint': 'Fields marked with * are required.',
  'clients.save': 'Save',
  'clients.added': 'Client added',
  'clients.saved': 'Changes saved',
  'clients.deleted': 'Client deleted',
  'clients.choose': '— select —',
  'clients.yes': 'yes',
  'col.fio': 'Full name', 'col.birthDate': 'Date of birth', 'col.passport': 'Passport', 'col.identification': 'ID number',
  'col.city': 'City of residence', 'col.mobile': 'Mobile phone', 'col.income': 'Income, BYN',
  'section.personal': 'Personal data', 'section.passport': 'Passport data',
  'section.contacts': 'Residence and contacts', 'section.social': 'Social status',
  'field.lastName': 'Last name', 'field.firstName': 'First name', 'field.middleName': 'Patronymic',
  'field.birthDate': 'Date of birth', 'field.sex': 'Sex', 'field.birthPlace': 'Place of birth',
  'field.passportSeries': 'Passport series', 'field.passportNumber': 'Passport No.',
  'field.identificationNumber': 'Identification number', 'field.issuedBy': 'Issued by', 'field.issueDate': 'Date of issue',
  'field.residenceCityId': 'City of residence', 'field.residenceAddress': 'Residential address',
  'field.registrationCityId': 'City of registration', 'field.homePhone': 'Home phone', 'field.mobilePhone': 'Mobile phone',
  'field.email': 'E-mail', 'field.maritalStatusId': 'Marital status', 'field.citizenshipId': 'Citizenship',
  'field.disabilityId': 'Disability', 'field.monthlyIncome': 'Monthly income, BYN', 'field.pensioner': 'Pensioner',
  'sex.M': 'Male', 'sex.F': 'Female',
  'placeholder.date': 'DD.MM.YYYY',
  'check.letters': 'Letters only',
  'check.dateInvalid': 'No such date, format DD.MM.YYYY',
  'check.dateFuture': 'The date cannot be in the future',
  'check.dateOld': 'The date cannot be earlier than 1900',
  'check.passportSeries': 'Series: two capital Latin letters',
  'check.passportNumber': 'Passport number: seven digits',
  'check.identificationNumber': 'Format: 7 digits, letter, 3 digits, 2 letters, digit',
  'check.issueBeforeBirth': 'The passport cannot be issued before the date of birth',
  'check.homePhone': 'Format: 293-88-44',
  'check.mobilePhone': 'Format: +375 (29) 314-15-92, code 25, 29, 33 or 44',
  'check.email': 'Invalid e-mail',
  'check.income': 'Amount: digits only, no more than 2 decimal places',
},
be: {
  'clients.page': 'Кліенты',
  'clients.list': 'Спіс кліентаў',
  'clients.count': 'Усяго кліентаў: {0}. Сартаванне па прозвішчы.',
  'clients.add': 'Дадаць кліента',
  'clients.empty': 'Кліентаў пакуль няма',
  'clients.edit': 'Змяніць',
  'clients.delete': 'Выдаліць',
  'clients.deleteClient': 'Выдаліць кліента',
  'clients.confirmDelete': 'Выдаліць кліента {0}?',
  'clients.new': 'Новы кліент',
  'clients.editing': 'Рэдагаванне кліента',
  'clients.requiredHint': 'Палі, пазначаныя *, абавязковыя для запаўнення.',

```

```

'clients.save': 'Захаваць',
'clients.added': 'Кліент дададзены',
'clients.saved': 'Змены захаваны',
'clients.deleted': 'Кліент выдалены',
'clients.choose': '— выберыце —',
'clients.yes': 'так',
'col.fio': 'ПІБ', 'col.birthDate': 'Дата нараджэння', 'col.passport': 'Пашпарт', 'col.identification': 'Ідэнт. нумар',
'col.city': 'Горад пражывання', 'col.mobile': 'Мабільны тэлефон', 'col.income': 'Даход, BYN',
'section.personal': 'Асабістыя даныя', 'section.passport': 'Пашпартныя даныя',
'section.contacts': 'Пражыванне і кантакты', 'section.social': 'Сацыяльны статус',
'field.lastName': 'Прозвішча', 'field.firstName': 'Імя', 'field.middleName': 'Імя па бацьку',
'field.birthDate': 'Дата нараджэння', 'field.sex': 'Пол', 'field.birthPlace': 'Месца нараджэння',
'field.passportSeries': 'Серыя пашпарта', 'field.passportNumber': '№ пашпарта',
'field.identificationNumber': 'Ідэнтыфікацыйны нумар', 'field.issuedBy': 'Кім выдадзены', 'field.issueDate': 'Дата выдачы',
'field.residenceCityId': 'Горад фактычнага пражывання', 'field.residenceAddress': 'Адрас фактычнага пражывання',
'field.registrationCityId': 'Горад праніскі', 'field.homePhone': 'Тэлефон хатні', 'field.mobilePhone': 'Тэлефон мабільны',
'field.email': 'Е-mail', 'field.maritalStatusId': 'Сямейнае становішча', 'field.citizenshipId': 'Грамадзянства',
'field.disabilityId': 'Інваліднасць', 'field.monthlyIncome': 'Штомесячны даход, BYN', 'field.pensioner': 'Пенсіянер',
'sex.M': 'Мужчынскі', 'sex.F': 'Жаночы',
'placeholder.date': 'ДД.ММ.ГГГГ',
'check.letters': 'Дапущальныя толькі літары',
'check.dateInvalid': 'Такой даты не існуе, фармат ДД.ММ.ГГГГ',
'check.dateFuture': 'Дата не можа быць у будучыні',
'check.dateOld': 'Дата не можа быць раней за 1900 год',
'check.passportSeries': 'Серыя — дзве вялікія лацінскія літары',
'check.passportNumber': 'Нумар пашпарта — сем лічбаў',
'check.identificationNumber': 'Фармат: 7 лічбаў, літара, 3 лічбы, 2 літары, лічба',
'check.issueBeforeBirth': 'Пашпарт не можа быць выдадзены раней за дату нараджэння',
'check.homePhone': 'Фармат: 293-88-44',
'check.mobilePhone': 'Фармат: +375 (29) 314-15-92, код 25, 29, 33 або 44',
'check.email': 'Некарэктны е-mail',
'check.income': 'Грашовая сума: толькі лічбы і не больш за 2 знакі пасля коскі',
},
});

```

Листинг A.50 — client-service/src/test/java/by/bsuir/bank/client/ClientApiIT.java

```

package by.bsuir.bank.client;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.MethodSource;
import org.junit.jupiter.params.provider.ValueSource;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.ResultActions;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Map;

import static org.assertj.core.api.Assertions.assertThat;
import static org.hamcrest.Matchers.contains;
import static org.hamcrest.Matchers.hasSize;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.put;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/**
 * Интеграционные тесты REST-сервиса на базе H2 в памяти: запрос идёт мимо web-клиента,
 * поэтому проверяется именно серверная валидация. После каждого запроса состояние
 * таблицы client проверяется SQL-запросом «прямо в базе».
 */
// своя база на каждый класс тестов: контексты не делят состояние
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@AutoConfigureMockMvc
@Transactional
class ClientApiIT {

    private static final int SEEDER = 6;

```

```

@Autowired
private MockMvc mvc;
@Autowired
private JdbcTemplate jdbc;

@Test
@DisplayName("База заполняется при старте, список отсортирован по фамилии")
void listIsSortedByLastName() throws Exception {
    mvc.perform(get("/api/clients"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$", hasSize(SEEDED)))
        .andExpect(jsonPath("$[*].lastName",
            contains("Абрамович", "Жуков", "Ковальчук", "Лебедев", "Савицкая", "Шевчук")));
}

@Test
@DisplayName("Справочники отдаются для выпадающих списков, городов не меньше пяти")
void dictionariesAreAvailable() throws Exception {
    mvc.perform(get("/api/dictionaries"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.cities.length()").value(8))
        .andExpect(jsonPath("$.maritalStatuses.length()").value(4))
        .andExpect(jsonPath("$.citizenships.length()").value(5))
        .andExpect(jsonPath("$.disabilities.length()").value(4));
}

@Test
@DisplayName("n.1 Повторный ввод тех же данных – ошибка, в базе один клиент")
void sameClientTwice() throws Exception {
    create(TestClients.valid()).andExpect(status().isCreated());

    create(TestClients.valid())
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.fields.passportNumber").exists())
        .andExpect(jsonPath("$.fields.identificationNumber").exists());

    assertThat(count("last_name = 'Иванов'")).isEqualTo(1);
}

@Test
@DisplayName("n.2 Другой клиент с тем же паспортом – ошибка, паспорт в базе один")
void anotherClientWithSamePassport() throws Exception {
    create(TestClients.valid()).andExpect(status().isCreated());

    Map<String, Object> other = TestClients.valid();
    other.put("lastName", "Петров");
    other.put("firstName", "Пётр");
    other.put("identificationNumber", "3010185A001PB9");
    create(other)
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.fields.passportSeries").exists())
        .andExpect(jsonPath("$.fields.passportNumber").exists())
        .andExpect(jsonPath("$.fields.identificationNumber").doesNotExist());

    assertThat(count("passport_series = 'MP' and passport_number = '7654321'")).isEqualTo(1);
    assertThat(count("last_name = 'Петров'")).isZero();
}

@Test
@DisplayName("n.3 Другой клиент с тем же идентификационным номером – ошибка")
void anotherClientWithSameIdentificationNumber() throws Exception {
    create(TestClients.valid()).andExpect(status().isCreated());

    Map<String, Object> other = TestClients.valid();
    other.put("lastName", "Петров");
    other.put("passportNumber", "1111111");
    create(other)
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.fields.identificationNumber").exists())
        .andExpect(jsonPath("$.fields.passportNumber").doesNotExist());

    assertThat(count("identification_number = '3170590A077PB4'")).isEqualTo(1);
}

@Test
@DisplayName("n.4 Фамилия «1234» – ошибка, клиента 1234 в базе нет")
void digitsInsteadOfLastName() throws Exception {
    create(TestClients.with("lastName", "1234"))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields.lastName").value("Допустимы только буквы"));

    assertThat(count("last_name = '1234'")).isZero();
}

```

```

    }

    @Test
    @DisplayName("н.5 Пробел вместо имени – ошибка, клиент не записан")
    void spaceInsteadOfFirstName() throws Exception {
        create(TestClients.with("firstName", " "))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.fields.firstName").value("Обязательное поле"));

        assertThat(count("last_name = 'Иванов'")).isZero();
    }

    @ParameterizedTest(name = "дата рождения «{0}»")
    @ValueSource(strings = {"не дата", "31.02.2015", "29.02.2015"})
    @DisplayName("н.6 Текст, не являющийся датой, – ошибка")
    void birthDateIsNotDate(String birthDate) throws Exception {
        create(TestClients.with("birthDate", birthDate))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.fields.birthDate").exists());

        assertThat(count("last_name = 'Иванов'")).isZero();
    }

    @ParameterizedTest(name = "{0} = «{1}»")
    @CsvSource(delimiter = '|', value = {
        "passportSeries|mp",
        "passportNumber|12345",
        "identificationNumber|3170590-077PB4",
        "homePhone|2014567",
        "mobilePhone|+375 (17) 765-43-21",
        "issueDate|30.02.2018",
        "email|ivanov@"
    })
    @DisplayName("н.6 Поля с маской проверяются на сервере")
    void maskedFieldsAreValidatedByService(String field, String value) throws Exception {
        create(TestClients.with(field, value))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.fields." + field).exists());

        assertThat(count("last_name = 'Иванов'")).isZero();
    }

    @ParameterizedTest(name = "без поля {0}")
    @MethodSource("requiredFields")
    @DisplayName("н.7 Без любого обязательного поля запись не выполняется")
    void requiredFieldIsMissing(String field) throws Exception {
        Map<String, Object> client = TestClients.valid();
        client.remove(field);

        create(client)
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.fields." + field).value("Обязательное поле"));

        assertThat(count("1 = 1")).isEqualTo(SEEDED);
    }

    static List<String> requiredFields() {
        return TestClients.REQUIRED_FIELDS;
    }

    @Test
    @DisplayName("н.8 Только обязательные поля – клиент записан")
    void onlyRequiredFields() throws Exception {
        create(TestClients.onlyRequired())
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").isNumber())
            .andExpect(jsonPath("$.fullName").value("Иванов Иван Иванович"))
            .andExpect(jsonPath("$.mobilePhone").isEmpty())
            .andExpect(jsonPath("$.monthlyIncome").isEmpty());

        assertThat(count("last_name = 'Иванов' and home_phone is null and email is null")).isEqualTo(1);
    }

    @Test
    @DisplayName("Текст в денежном поле – ошибка разбора запроса с указанием поля")
    void textInMoneyField() throws Exception {
        create(TestClients.with("monthlyIncome", "много"))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.fields.monthlyIncome").exists());
    }

    @Test

```

```

@DisplayName("Редактирование: изменения сохраняются, чужой паспорт занять нельзя")
void update() throws Exception {
    long id = jdbc.queryForObject("select id from client where last_name = 'Жуков'", Long.class);

    Map<String, Object> changed = TestClients.valid();
    changed.put("lastName", "Жуков-Новый");
    mvc.perform(put("/api/clients/" + id).contentType(MediaType.APPLICATION_JSON).content(TestClients.json(changed)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.lastName").value("Жуков-Новый"));
    assertThat(count("last_name = 'Жуков-Новый' and passport_number = '7654321'")).isEqualTo(1);

    // паспорт клиента Лебедев (MP 3141592) уже занят
    changed.put("passportNumber", "3141592");
    mvc.perform(put("/api/clients/" + id).contentType(MediaType.APPLICATION_JSON).content(TestClients.json(changed)))
        .andExpect(status().isConflict());
}

@Test
@DisplayName("Удаление клиента")
void remove() throws Exception {
    long id = jdbc.queryForObject("select id from client where last_name = 'Шевчук'", Long.class);

    mvc.perform(delete("/api/clients/" + id)).andExpect(status().isNoContent());
    assertThat(count("1 = 1")).isEqualTo(SEEDED - 1);

    mvc.perform(delete("/api/clients/" + id)).andExpect(status().isNotFound());
    mvc.perform(get("/api/clients/" + id)).andExpect(status().isNotFound());
}

private ResultActions create(Map<String, Object> client) throws Exception {
    return mvc.perform(post("/api/clients").contentType(MediaType.APPLICATION_JSON).content(TestClients.json(client)));
}

/** Проверка «прямо в базе»: SQL-запрос к таблице client в обход сервиса и репозитория. */
private int count(String where) {
    return jdbc.queryForObject("select count(*) from client where " + where, Integer.class);
}
}

```

Листинг А.51 – client-service/src/test/java/by/bsuir/bank/client/ClientLanguageIT.java

```

package by.bsuir.bank.client;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.transaction.annotation.Transactional;

import static org.hamcrest.Matchers.hasItem;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/** Интеграционные тесты локализации: язык ответа задаётся заголовком Accept-Language. */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@AutoConfigureMockMvc
@Transactional
class ClientLanguageIT {

    @Autowired
    private MockMvc mvc;

    @ParameterizedTest(name = "{0}: {1}, {2}")
    @CsvSource({
        "ru, Обязательное поле, Допустимы только буквы",
        "en, Required field, Letters only",
        "be, Абавязковае поле, Далушчальныя толькі літары",
        "en-US, Required field, Letters only",
        "fr, Обязательное поле, Допустимы только буквы",
    })
    @DisplayName("Сообщения валидации приходят на языке запроса, неизвестный язык заменяется русским")
}

```

```

void validationMessages(String language, String required, String lettersOnly) throws Exception {
    var client = TestClients.with("lastName", "1234");
    client.remove("firstName");

    mvc.perform(post("/api/clients").header(HttpHeaders.ACCEPT_LANGUAGE, language)
        .contentType(MediaType.APPLICATION_JSON).content(TestClients.json(client)))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields.firstName").value(required))
        .andExpect(jsonPath("$.fields.lastName").value(lettersOnly));
}

@ParameterizedTest(name = "{0}: {1}")
@CsvSource({
    "ru, Минск, Женат / замужем, Республика Беларусь",
    "en, Minsk, Married, Republic of Belarus",
    "be, Мінск, Жанаты / замужам, Рэспубліка Беларусь",
})
@DisplayName("Справочники отдаются на языке запроса")
void dictionaries(String language, String city, String maritalStatus, String citizenship) throws Exception {
    mvc.perform(get("/api/dictionaries").header(HttpHeaders.ACCEPT_LANGUAGE, language))
        .andExpect(jsonPath("$.cities[*].name", hasItem(city)))
        .andExpect(jsonPath("$.maritalStatuses[*].name", hasItem(maritalStatus)))
        .andExpect(jsonPath("$.citizenships[0].name").value(citizenship))
        .andExpect(jsonPath("$.cities[0].nameEn").doesNotExist());
}

@Test
@DisplayName("Ошибка уникальности и данные клиента – на белорусском языке")
void businessMessages() throws Exception {
    var duplicate = TestClients.with("passportNumber", "3141592");

    mvc.perform(post("/api/clients").header(HttpHeaders.ACCEPT_LANGUAGE, "be")
        .contentType(MediaType.APPLICATION_JSON).content(TestClients.json(duplicate)))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.message").value("Такі кліент ужо ёсць у базе"))
        .andExpect(jsonPath("$.fields.passportNumber").value("Кліент з такім пашпартам ужо зарэгістраваны"));

    mvc.perform(get("/api/clients").header(HttpHeaders.ACCEPT_LANGUAGE, "be"))
        .andExpect(jsonPath("$.residenceCity").value("Гродна"))
        .andExpect(jsonPath("$.lastName").value("Абрамович"));
}
}

```

Листинг А.52 –

client-service/src/test/java/by/bsuir/bank/client/ClientRequestValidationTest.java

```

package by.bsuir.bank.client;

import by.bsuir.bank.client.dto.ClientRequest;
import jakarta.validation.ConstraintViolation;
import jakarta.validation.Validation;
import jakarta.validation.Validator;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.MethodSource;

import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.stream.Collectors;

import static org.assertj.core.api.Assertions.assertThat;

/** Модульные тесты серверной валидации формы: маски и обязательность полей (пп. 4–8 программы защиты). */
class ClientRequestValidationTest {

    private final Validator validator = Validation.buildDefaultValidatorFactory().getValidator();

    @Test
    void validFormHasNoViolations() {
        assertThat(validator.validate(TestClients.valid()).isEmpty());
    }

    @Test
    void optionalFieldsMayBeOmittedOrEmpty() {
        assertThat(validator.validate(TestClients.onlyRequired()).isEmpty());
    }

    Map<String, Object> blank = TestClients.valid();
    blank.put("homePhone", "");
}

```

```

        blank.put("mobilePhone", " ");
        blank.put("email", "");
        blank.put("monthlyIncome", null);
        assertThat(invalidFields(blank)).isEmpty();
    }

    @ParameterizedTest(name = "без поля {0}")
    @MethodSource("requiredFields")
    void everyRequiredFieldIsChecked(String field) {
        assertThat(invalidFields(TestClients.with(field, null))).containsExactly(field);
    }

    static List<String> requiredFields() {
        return TestClients.REQUIRED_FIELDS;
    }

    @ParameterizedTest(name = "{0} = «{1}»")
    @CsvSource(delimiter = '|', ignoreLeadingAndTrailingWhitespace = false, value = {
        "lastName|1234",
        "lastName|Иванов2",
        "lastName|Иванов-",
        "firstName| ",
        "firstName|Иван!",
        "middleName|_",
        "sex|X",
        "passportSeries|mp",
        "passportSeries|M1",
        "passportSeries|MPP",
        "passportNumber|123456",
        "passportNumber|12345678",
        "passportNumber|12A4567",
        "identificationNumber|3170590A077PB",
        "identificationNumber|3170590a077pb4",
        "identificationNumber|31705900077PB4",
        "homePhone|2014567",
        "homePhone|201-45-6",
        "mobilePhone|+375 (17) 765-43-21",
        "mobilePhone|80297654321",
        "mobilePhone|+375 (29) 765-43-2",
        "email|ivanov@",
        "email|ivanov.example.by",
        "birthDate|29.02.2015",
        "issueDate|31.02.2015",
    })
    void maskedFieldsRejectWrongFormat(String field, String value) {
        assertThat(invalidFields(TestClients.with(field, value))).containsExactly(field);
    }

    @ParameterizedTest
    @CsvSource({"-1", "0.001", "1234567890123"})
    void monthlyIncomeIsMoney(String income) {
        assertThat(invalidFields(TestClients.with("monthlyIncome", income))).containsExactly("monthlyIncome");
    }

    @Test
    void compositeNamesAreAllowed() {
        Map<String, Object> client = TestClients.valid();
        client.put("lastName", "Петрова-Водкина");
        client.put("firstName", "Анна Мария");
        client.put("middleName", "Ивановна ");
        ClientRequest request = TestClients.request(client);
        assertThat(validator.validate(request)).isEmpty();
        assertThat(request.middleName()).isEqualTo("Ивановна");
    }

    private Set<String> invalidFields(Map<String, Object> fields) {
        return validator.validate(TestClients.request(fields)).stream()
            .map(ConstraintViolation::getPropertyPath)
            .map(Object::toString)
            .collect(Collectors.toSet());
    }
}

```

Листинг А.53 – client-service/src/test/java/by/bsuir/bank/client/ClientServiceTest.java

```

package by.bsuir.bank.client;

import by.bsuir.bank.client.domain.Citizenship;
import by.bsuir.bank.client.domain.City;

```

```

import by.bsuir.bank.client.domain.Client;
import by.bsuir.bank.client.domain.Disability;
import by.bsuir.bank.client.domain.MaritalStatus;
import by.bsuir.bank.client.dto.ClientResponse;
import by.bsuir.bank.client.repository.CitizenshipRepository;
import by.bsuir.bank.client.repository.CityRepository;
import by.bsuir.bank.client.repository.ClientRepository;
import by.bsuir.bank.client.repository.DisabilityRepository;
import by.bsuir.bank.client.repository.MaritalStatusRepository;
import by.bsuir.bank.client.service.ClientService;
import by.bsuir.bank.client.service.FieldErrorException;
import by.bsuir.bank.client.service.NotFoundException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.mockito.junit.jupiter.MockitoSettings;
import org.mockito.quality.Strictness;
import org.springframework.http.HttpStatus;

import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.anyLong;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

/** Модульные тесты бизнес-правил сервиса на заглушках репозиториях (пп. 1–3 программы защиты). */
@ExtendWith(MockitoExtension.class)
@MockitoSettings(strictness = Strictness.LENIENT)
class ClientServiceTest {

    @Mock
    private ClientRepository clients;
    @Mock
    private CityRepository cities;
    @Mock
    private MaritalStatusRepository maritalStatuses;
    @Mock
    private CitizenshipRepository citizenships;
    @Mock
    private DisabilityRepository disabilities;

    @InjectMocks
    private ClientService service;

    @BeforeEach
    void dictionaries() {
        when(cities.findById(anyLong())).thenReturn(Optional.of(named(new City(), "Минск")));
        when(maritalStatuses.findById(anyLong())).thenReturn(Optional.of(named(new MaritalStatus(), "Холост")));
        when(citizenships.findById(anyLong())).thenReturn(Optional.of(named(new Citizenship(), "Республика Беларусь")));
        when(disabilities.findById(anyLong())).thenReturn(Optional.of(named(new Disability(), "Нет")));
        when(clients.saveAndFlush(any(Client.class))).thenAnswer(call -> {
            Client client = call.getArgument(0);
            client.setId(100L);
            return client;
        });
    }

    @Test
    void savesNewClient() {
        ClientResponse saved = service.create(TestClients.request(TestClients.valid()));

        assertThat(saved.id()).isEqualTo(100L);
        assertThat(saved.fullName()).isEqualTo("Иванов Иван Иванович");
        assertThat(saved.birthDate()).isEqualTo("17.05.1990");
        assertThat(saved.residenceCity()).isEqualTo("Минск");
    }

    @Test
    void emptyOptionalFieldsAreStoredAsNull() {
        var fields = TestClients.valid();
        fields.put("homePhone", "");
        fields.put("email", " ");

        ClientResponse saved = service.create(TestClients.request(fields));
    }

```

```

        assertThat(saved.homePhone()).isNull();
        assertThat(saved.email()).isNull();
    }

    @Test
    void rejectsSecondClientWithSamePassport() {
        when(clients.findByPassportSeriesAndPassportNumber("MP", "7654321")).thenReturn(Optional.of(existing(7L)));

        assertThatThrownBy(() -> service.create(TestClients.request(TestClients.valid())))
            .isInstanceOfSatisfying(FieldErrorException.class, e -> {
                assertThat(e.getStatus()).isEqualTo(HttpStatus.CONFLICT);
                assertThat(e.getFields()).containsOnlyKeys("passportSeries", "passportNumber");
            });
        verify(clients, never()).saveAndFlush(any());
    }

    @Test
    void rejectsSecondClientWithSameIdentificationNumber() {
        when(clients.findByIdentificationNumber("3170590A077PB4")).thenReturn(Optional.of(existing(7L)));

        assertThatThrownBy(() -> service.create(TestClients.request(TestClients.valid())))
            .isInstanceOfSatisfying(FieldErrorException.class,
                e -> assertThat(e.getFields()).containsOnlyKeys("identificationNumber"));
        verify(clients, never()).saveAndFlush(any());
    }

    @Test
    void clientMayKeepOwnPassportOnUpdate() {
        Client self = existing(7L);
        when(clients.findById(7L)).thenReturn(Optional.of(self));
        when(clients.findByPassportSeriesAndPassportNumber("MP", "7654321")).thenReturn(Optional.of(self));
        when(clients.findByIdentificationNumber("3170590A077PB4")).thenReturn(Optional.of(self));

        service.update(7L, TestClients.request(TestClients.with("lastName", "Петров")));

        assertThat(self.getLastName()).isEqualTo("Петров");
        verify(clients).saveAndFlush(self);
    }

    @Test
    void passportCannotBeIssuedBeforeBirth() {
        var fields = TestClients.with("issueDate", "16.05.1990");

        assertThatThrownBy(() -> service.create(TestClients.request(fields)))
            .isInstanceOfSatisfying(FieldErrorException.class,
                e -> assertThat(e.getFields()).containsOnlyKeys("issueDate"));
    }

    @Test
    void rejectsValueMissingInDictionary() {
        when(cities.findById(99L)).thenReturn(Optional.empty());

        assertThatThrownBy(() -> service.create(TestClients.request(TestClients.with("residenceCityId", 99))))
            .isInstanceOfSatisfying(FieldErrorException.class,
                e -> assertThat(e.getFields()).containsOnlyKeys("residenceCityId"));
    }

    @Test
    void deletingUnknownClientFails() {
        when(clients.existsById(5L)).thenReturn(false);

        assertThatThrownBy(() -> service.delete(5L)).isInstanceOf(NotFoundException.class);
        verify(clients, never()).deleteById(any());
    }

    private static Client existing(Long id) {
        Client client = new Client();
        client.setId(id);
        return client;
    }

    private static <T extends by.bsuir.bank.client.domain.DictionaryItem> T named(T item, String name) {
        item.setId(1L);
        item.setName(name);
        return item;
    }
}

```

Листинг А.54 – client-service/src/test/java/by/bsuir/bank/client/ClientUiIT.java

```

package by.bsuir.bank.client;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.Assumptions;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.server.LocalServerPort;
import org.springframework.jdbc.core.JdbcTemplate;

import java.time.Duration;
import java.util.List;
import java.util.Map;

import static org.assertj.core.api.Assertions.assertThat;

/**
 * Функциональные тесты через WebDriver: настоящий браузер (Chrome без окна) открывает web-клиент
 * поднятого сервиса и проходит программу защиты. Результат каждого шага сверяется с таблицей client в H2.
 */
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
    properties = {"spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1"})
class ClientUiIT {

    private static WebDriver driver;

    @LocalServerPort
    private int port;
    @Autowired
    private JdbcTemplate jdbc;

    private WebDriverWait wait;

    @BeforeAll
    static void startBrowser() {
        try {
            ChromeOptions options = new ChromeOptions();
            options.addArguments("--headless=new", "--window-size=1400,1600", "--lang=ru");
            driver = new ChromeDriver(options);
        } catch (Exception e) {
            Assumptions.abort("Chrome недоступен, функциональные тесты пропущены: " + e.getMessage());
        }
    }

    @AfterAll
    static void stopBrowser() {
        if (driver != null) {
            driver.quit();
        }
    }

    @BeforeEach
    void openNewClientForm() {
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        driver.get("http://localhost:" + port + "/");
        wait.until(ExpectedConditions.numberOfElementsToBeMoreThan(By.cssSelector("#clients tbody tr"), 0));
        driver.findElement(By.id("btn-add")).click();
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("btn-save")));
    }

    @AfterEach
    void removeCreatedClients() {
        jdbc.update("delete from client where id > 6");
    }

    @Test
    @DisplayName("Форма «Список клиентов» отсортирована по фамилии")

```

```

void listIsSortedByLastName() {
    driver.findElement(By.id("btn-cancel")).click();
    List<String> names = driver.findElements(By.cssSelector("#clients td.fio")).stream()
        .map(WebElement::getText).toList();
    assertThat(names).hasSize(6).isSorted();
    assertThat(names.get(3)).isEqualTo("Лебедев Артем Владимирович");
}

@Test
@DisplayName("н.1 Те же данные ещё раз — сообщение об ошибке, в базе один клиент")
void sameClientTwice() {
    fill(TestClients.valid());
    save();
    waitForNotice("Клиент добавлен");

    driver.findElement(By.id("btn-add")).click();
    fill(TestClients.valid());
    save();

    assertThat(formAlert()).isEqualTo("Такой клиент уже есть в базе");
    assertThat(error("passportNumber")).isEqualTo("Клиент с таким паспортом уже зарегистрирован");
    assertThat(count("last_name = 'Иванов'")).isEqualTo(1);
}

@Test
@DisplayName("н.2 Другой клиент с паспортом студента — сообщение об ошибке")
void anotherClientWithSamePassport() {
    Map<String, Object> other = TestClients.valid();
    other.put("passportSeries", "MP");
    other.put("passportNumber", "3141592"); // паспорт клиента Лебедев из начальных данных
    fill(other);
    save();

    assertThat(error("passportSeries")).isEqualTo("Клиент с таким паспортом уже зарегистрирован");
    assertThat(count("passport_series = 'MP' and passport_number = '3141592'")).isEqualTo(1);
    assertThat(count("last_name = 'Иванов'")).isZero();
}

@Test
@DisplayName("н.3 Другой клиент с идентификационным номером студента — сообщение об ошибке")
void anotherClientWithSameIdentificationNumber() {
    fill(TestClients.with("identificationNumber", "3140301A001PB5"));
    save();

    assertThat(error("identificationNumber")).isEqualTo("Клиент с таким идентификационным номером уже зарегистрирован");
    assertThat(count("identification_number = '3140301A001PB5'")).isEqualTo(1);
    assertThat(count("last_name = 'Иванов'")).isZero();
}

@Test
@DisplayName("н.4 Фамилия «1234» — сообщение об ошибке, в базе такой фамилии нет")
void digitsInsteadOfLastName() {
    fill(TestClients.with("lastName", "1234"));
    save();

    assertThat(error("lastName")).isEqualTo("Допустимы только буквы");
    assertThat(count("last_name = '1234'")).isZero();
}

@Test
@DisplayName("н.5 Пробел вместо имени — сообщение об ошибке, клиент не записан")
void spaceInsteadOfFirstName() {
    fill(TestClients.with("firstName", " "));
    save();

    assertThat(error("firstName")).isEqualTo("Обязательное поле");
    assertThat(count("last_name = 'Иванов'")).isZero();
}

@ParameterizedTest(name = "дата рождения «{0}»")
@CsvSource(delimiter = '|', value = {
    "не дата|Обязательное поле", // маска не пропускает буквы — поле остаётся пустым
    "31.02.2015|Такой даты не существует, формат ДД.ММ.ГГГГ",
    "29.02.2015|Такой даты не существует, формат ДД.ММ.ГГГГ",
    "15.07.20|Такой даты не существует, формат ДД.ММ.ГГГГ",
})
@Test
@DisplayName("н.6 В поле даты рождения текст, не являющийся датой")
void birthDateIsNotDate(String typed, String message) {
    fill(TestClients.with("birthDate", typed));
    save();

    assertThat(error("birthDate")).isEqualTo(message);
}

```

```

        assertThat(count("last_name = 'Иванов'")).isZero();
    }

    @ParameterizedTest(name = "{0}: «{1}»")
    @CsvSource(delimiter = '|', value = {
        "passportSeries|M|Серия – две заглавные латинские буквы",
        "passportNumber|12345|Номер паспорта – семь цифр",
        "identificationNumber|3170590A077|Формат: 7 цифр, буква, 3 цифры, 2 буквы, цифра",
        "homePhone|20145|Формат: 293-88-44",
        "mobilePhone|+375 (17) 765-43-21|Формат: +375 (29) 314-15-92, код 25, 29, 33 или 44",
        "issueDate|30.02.2018|Такой даты не существует, формат ДД.ММ.ГГГГ",
    })
    @DisplayName("n.6 Поля с маской: неполное или неверное значение")
    void maskedFields(String field, String typed, String message) {
        fill(TestClients.with(field, typed));
        save();

        assertThat(error(field)).isEqualTo(message);
        assertThat(count("last_name = 'Иванов'")).isZero();
    }

    @Test
    @DisplayName("Маска ввода отбрасывает недопустимые символы и расставляет разделители")
    void masksFilterInput() {
        type("passportNumber", "12ab34567999");
        type("passportSeries", "mp1");
        type("birthDate", "17051990");
        type("mobilePhone", "297654321");

        assertThat(value("passportNumber")).isEqualTo("1234567");
        assertThat(value("passportSeries")).isEqualTo("MP");
        assertThat(value("birthDate")).isEqualTo("17.05.1990");
        assertThat(value("mobilePhone")).isEqualTo("+375 (29) 765-43-21");
    }

    @ParameterizedTest(name = "без поля {0}")
    @ValueSource(strings = {
        "lastName", "firstName", "middleName", "birthDate", "sex", "passportSeries", "passportNumber",
        "issuedBy", "issueDate", "identificationNumber", "birthPlace", "residenceCityId", "residenceAddress",
        "registrationCityId", "maritalStatusId", "citizenshipId", "disabilityId"})
    @DisplayName("n.7 Не заполнено одно обязательное поле – сообщение об ошибке, записи нет")
    void requiredFieldIsMissing(String field) {
        Map<String, Object> client = TestClients.valid();
        client.remove(field);
        fill(client);
        save();

        assertThat(error(field)).isEqualTo("Обязательное поле");
        assertThat(count("1 = 1")).isEqualTo(6);
    }

    @Test
    @DisplayName("n.8 Только обязательные поля – клиент записан и появился в списке")
    void onlyRequiredFields() {
        fill(TestClients.onlyRequired());
        save();
        waitForNotice("Клиент добавлен");

        assertThat(driver.findElement(By.id("clients")).getText()).contains("Иванов Иван Иванович");
        assertThat(count("last_name = 'Иванов' and mobile_phone is null and monthly_income is null")).isEqualTo(1);
    }

    @Test
    @DisplayName("Редактирование и удаление клиента через web-клиент")
    void editAndDelete() {
        fill(TestClients.valid());
        save();
        waitForNotice("Клиент добавлен");

        row("Иванов Иван Иванович").findElement(By.cssSelector("button.edit")).click();
        wait.until(ExpectedConditions.attributeToBe(By.id("f-lastName"), "value", "Иванов"));
        assertThat(value("monthlyIncome")).isEqualTo("2500.50");
        type("lastName", "Сидоров");
        save();
        waitForNotice("Изменения сохранены");
        assertThat(count("last_name = 'Сидоров' and passport_number = '7654321'")).isEqualTo(1);

        row("Сидоров Иван Иванович").findElement(By.cssSelector("button.delete")).click();
        wait.until(ExpectedConditions.alertIsPresent()).accept();
        waitForNotice("Клиент удалён");
        assertThat(count("last_name = 'Сидоров'")).isZero();
    }
}

```

```

@Test
@DisplayName("Переключение языка интерфейса: русский, английский, белорусский")
void languageSwitch() {
    try {
        driver.findElement(By.id("lang-en")).click();
        wait.until(ExpectedConditions.textToBe(By.cssSelector("h1"), "Client list"));
        assertThat(driver.getTitle()).isEqualTo("Clients – BankEt");
        driver.findElement(By.id("btn-add")).click();
        save();
        assertThat(error("lastName")).isEqualTo("Required field");
        assertThat(driver.findElement(By.cssSelector("label[for=f-lastName]")).getText()).startsWith("Last name");
        assertThat(new Select(driver.findElement(By.id("f-residenceCityId"))).getOptions())
            .extracting(WebElement::getText).contains("Minsk", "Grodno");

        // сообщение сервера тоже приходит на выбранном языке
        fill(TestClients.with("passportNumber", "3141592"));
        save();
        assertThat(error("passportNumber")).isEqualTo("A client with this passport is already registered");

        driver.findElement(By.id("lang-be")).click();
        wait.until(ExpectedConditions.textToBe(By.cssSelector("h1"), "Спіс кліентаў"));
        assertThat(driver.findElement(By.id("btn-add")).getText()).isEqualTo("Дадаць кліента");
    } finally {
        driver.manage().deleteCookieNamed("bank_lang"); // остальные тесты работают на русском
    }
}

// ----- работа со страницей -----

private void fill(Map<String, Object> client) {
    client.forEach((field, value) -> {
        switch (field) {
            case "sex" -> driver.findElement(By.id("f-sex-" + value)).click();
            case "pensioner" -> {
                if (Boolean.TRUE.equals(value)) {
                    driver.findElement(By.id("f-pensioner")).click();
                }
            }
            case "residenceCityId", "registrationCityId", "maritalStatusId", "citizenshipId", "disabilityId" ->
                new Select(driver.findElement(By.id("f-" + field))).selectByIndex((Integer) value);
            default -> type(field, String.valueOf(value));
        }
    });
}

private void type(String field, String text) {
    WebElement input = driver.findElement(By.id("f-" + field));
    input.clear();
    input.sendKeys(text);
}

private String value(String field) {
    return driver.findElement(By.id("f-" + field)).getDomProperty("value");
}

private void save() {
    driver.findElement(By.id("btn-save")).click();
}

private String error(String field) {
    return wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("e-" + field))).getText();
}

private String formAlert() {
    return wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("form-alert"))).getText();
}

private void waitForNotice(String text) {
    wait.until(ExpectedConditions.textToBe(By.id("notice"), text));
}

private WebElement row(String fullName) {
    return driver.findElement(By.xpath("//table[@id='clients']/tr[td[normalize-space()=' " + fullName + "']]"));
}

/** Проверка «прямо в базе»: SQL-запрос к таблице client. */
private int count(String where) {
    return jdbc.queryForObject("select count(*) from client where " + where, Integer.class);
}
}

```

Листинг А.55 –

client-service/src/test/java/by/bsuir/bank/client/DateTextValidatorTest.java

```
package by.bsuir.bank.client;

import by.bsuir.bank.client.validation.DateTextValidator;
import by.bsuir.bank.client.validation.Dates;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

import java.time.LocalDate;

import static org.assertj.core.api.Assertions.assertThat;

/** Модульные тесты проверки дат: п. 6 программы защиты. */
class DateTextValidatorTest {

    private final DateTextValidator validator = new DateTextValidator();

    @ParameterizedTest
    @ValueSource(strings = {"14.03.2001", "29.02.2016", "01.01.1900", "31.12.1999"})
    void acceptsExistingDates(String text) {
        assertThat(validator.isValid(text, null)).isTrue();
    }

    @ParameterizedTest
    @ValueSource(strings = {
        "не дата", "31.02.2015", "29.02.2015", "32.01.2000", "00.10.2000", "15.13.2000",
        "2015-02-28", "1.2.2015", "28.02.15", "31.12.1899"})
    void rejectsTextThatIsNotDate(String text) {
        assertThat(validator.isValid(text, null)).isFalse();
    }

    @Test
    void rejectsFutureDate() {
        String tomorrow = Dates.format(LocalDate.now().plusDays(1));
        assertThat(validator.isValid(tomorrow, null)).isFalse();
        assertThat(validator.isValid(Dates.format(LocalDate.now()), null)).isTrue();
    }

    @Test
    void emptyValueIsLeftToNotBlank() {
        assertThat(validator.isValid(null, null)).isTrue();
        assertThat(validator.isValid(" ", null)).isTrue();
    }
}
```

Листинг А.56 – client-service/src/test/java/by/bsuir/bank/client/TestClients.java

```
package by.bsuir.bank.client;

import by.bsuir.bank.client.dto.ClientRequest;
import com.fasterxml.jackson.databind.ObjectMapper;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** Тестовые данные: корректно заполненная форма клиента, которую тесты портят по одному полю. */
public final class TestClients {

    public static final List<String> REQUIRED_FIELDS = List.of(
        "lastName", "firstName", "middleName", "birthDate", "sex", "passportSeries", "passportNumber",
        "issuedBy", "issueDate", "identificationNumber", "birthPlace", "residenceCityId", "residenceAddress",
        "registrationCityId", "maritalStatusId", "citizenshipId", "disabilityId", "pensioner");

    public static final List<String> OPTIONAL_FIELDS = List.of("homePhone", "mobilePhone", "email", "monthlyIncome");

    private static final ObjectMapper MAPPER = new ObjectMapper();

    /** Новый клиент, которого нет в начальном заполнении базы. */
    public static Map<String, Object> valid() {
        Map<String, Object> client = new LinkedHashMap<>();
        client.put("lastName", "Иванов");
        client.put("firstName", "Иван");
        client.put("middleName", "Иванович");
        client.put("birthDate", "17.05.1990");
        client.put("sex", "М");
        client.put("passportSeries", "MP");
    }
}
```

```

        client.put("passportNumber", "7654321");
        client.put("issuedBy", "Фрунзенское РУВД г. Минска");
        client.put("issueDate", "20.06.2018");
        client.put("identificationNumber", "3170590A077PB4");
        client.put("birthPlace", "г. Минск");
        client.put("residenceCityId", 1);
        client.put("residenceAddress", "ул. Притыцкого, д. 29, кв. 3");
        client.put("homePhone", "201-45-67");
        client.put("mobilePhone", "+375 (29) 765-43-21");
        client.put("email", "ivanov@example.by");
        client.put("registrationCityId", 1);
        client.put("maritalStatusId", 1);
        client.put("citizenshipId", 1);
        client.put("disabilityId", 1);
        client.put("pensioner", false);
        client.put("monthlyIncome", 2500.50);
        return client;
    }

    public static Map<String, Object> with(String field, Object value) {
        Map<String, Object> client = valid();
        client.put(field, value);
        return client;
    }

    public static Map<String, Object> onlyRequired() {
        Map<String, Object> client = valid();
        OPTIONAL_FIELDS.forEach(client::remove);
        return client;
    }

    public static ClientRequest request(Map<String, Object> fields) {
        return MAPPER.convertValue(fields, ClientRequest.class);
    }

    public static String json(Map<String, Object> fields) throws Exception {
        return MAPPER.writeValueAsString(fields);
    }

    private TestClients() {
    }
}

```